

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Bachelor of Technology / Computer Applications Final Year Major Project

A COMPREHENSIVE PROJECT REPORT ON EVENT PLANNING SYSTEM

(EventHub — Real-Time Campus Event Management & Ticketing
Platform)



Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science & Engineering

Submitted By (Project Team):

1. **Annu Maurya** (Roll No:
2201CSE001)
 2. **Anchal Pandey** (Roll No:
2201CSE002)
 3. **Krishna Ojha** (Roll No:
2201CSE003)
- B.Tech (CSE) Final Year*

Under the Guidance of:

Dr. / Prof. [Project Guide Name]
Assistant Professor
Department of Computer Science &
Engg.

ACADEMIC SESSION: 2025 – 2026

CERTIFICATE OF APPROVAL

This is to certify that the major project report entitled "**EVENT PLANNING SYSTEM (EVENTHUB)**" submitted by the following candidates in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** is an authentic record of bonafide project work carried out under my supervision and guidance:

1. **Annu Maurya** (Roll No: 2201CSE001)
2. **Anchal Pandey** (Roll No: 2201CSE002)
3. **Krishna Ojha** (Roll No: 2201CSE003)

To the best of my knowledge and belief, the matter embodied in this project report has not been submitted to any other University or Institution for the award of any degree or diploma. The system developed exhibits comprehensive full-stack software engineering principles, rigorous database design, and end-to-end operational functionality.

Date: _____

Place: _____

[Project Guide Name]

Project Guide & Supervisor
Department of CSE

Head of Department

Department of Computer Science & Engg.

External Examiner

Board of Examination

CANDIDATE'S DECLARATION

We, **Annu Maurya**, **Anchal Pandey**, and **Krishna Ojha**, students of Bachelor of Technology in Computer Science & Engineering, hereby declare that the work presented in this project report entitled "**EVENT PLANNING SYSTEM (EVENTHUB)**" is our own original work conducted under the guidance and supervision of our faculty advisor.

We confirm that:

- The work has been performed and implemented as part of our academic curriculum requirements.
- We have adhered strictly to standard academic integrity guidelines, and no part of this report has been plagiarized.
- All external libraries, open-source modules, architectural references, and data sources have been duly recognized and cited in the References and Bibliography chapters.
- This report has not been submitted concurrently for any other degree, diploma, or qualification to any other educational institution.

Annu Maurya

Roll No: 2201CSE001

B.Tech (CSE) Final Year

Anchal Pandey

Roll No: 2201CSE002

B.Tech (CSE) Final Year

Krishna Ojha

Roll No: 2201CSE003

B.Tech (CSE) Final Year

ACKNOWLEDGEMENT

The successful completion of any substantial engineering undertaking requires the guidance, support, and encouragement of numerous individuals. We wish to express our heartfelt gratitude to everyone who contributed to the realization of the **Event Planning System (EventHub)**.

First and foremost, we express our profound gratitude to our esteemed Project Guide, whose constant mentorship, technical discernment, and constructive critiques kept our efforts aligned with modern software engineering standards. Their guidance in architectural modularity, database indexing, and user experience design was instrumental to our success.

We also convey our sincere thanks to the Head of Department, Computer Science & Engineering, for fostering an academically stimulating environment and providing state-of-the-art computational laboratories, high-speed cloud access, and development infrastructure.

We extend our appreciation to all faculty members and technical staff of the department for their continuous support during lab sessions and testing phases.

Finally, we owe an eternal debt of gratitude to our parents and family members for their unwavering moral encouragement, patience, and sacrifices throughout our academic journey. We also thank our peers and classmates whose feedback during beta testing helped refine the EventHub interface.

Annu Maurya

Anchal Pandey

Krishna Ojha

Department of Computer Science & Engineering

ABSTRACT

In contemporary higher education institutions, organizing, publicizing, and participating in college events — including national hackathons, cultural festivals, sports tournaments, technical seminars, and entrepreneurship summits — is routinely crippled by fragmented communication channels, manual paper registrations, lack of dynamic seat tracking, and absence of unified attendee audit tools for organizers.

The **Event Planning System (EventHub)** is an enterprise-grade full-stack web application architected using the **MERN (MongoDB, Express.js, React.js, Node.js)** stack to transform campus event management into an automated, real-time, paperless ecosystem. The primary innovations and capabilities of EventHub include:

- **Cryptographic Role-Based Access Control (RBAC):** Employs JSON Web Tokens (JWT) and salted bcrypt password hashing to distinguish student participants from authorized organizers and college administrators.
- **Live MongoDB Event Inventory:** Features debounced search (300ms), 8-category pill filtering (Technology, Cultural, Sports, Business, Art, Music, Food), and multi-criteria sorting (Date, Popularity, Free First, Newest).
- **Concurrency & Seat Quota Guard:** Intelligently locks unauthenticated visitors out of registrations, enforces dynamic seat limit calculations, and blocks registrations once maximum capacity is reached.
- **Event Creation Studio with Live Preview:** Empowers authorized students to publish events with real-time card previewing, calendar validation prohibiting past-date scheduling (`min=Today`), and universal social image link normalization (Instagram post resolution, Google Drive public shares, CDN media).
- **In-Place Dynamic Event Modification:** Enables creators and administrators to adjust event dates, times, venues, and descriptions with immediate database synchronization.
- **Administrative Intelligence & 1-Click CSV Export:** Furnishes administrators with high-level platform analytics, attendee directories, and instantaneous CSV roster downloads for offline credentialing.
- **Ultra-Responsive Glassmorphism Interface:** Delivers an ergonomic dark purple aesthetic optimized across mobile viewports, tablets, and desktop displays.

Rigorous unit, integration, and user-acceptance testing confirmed zero overbooking, sub-100ms API query latencies, and 100% adherence to modern web standards.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 7.1	Three-Tier System Architecture of EventHub	32
Figure 7.2	Unified System Use Case Model	34
Figure 7.3	Sequence Diagram: User Authentication & Token Handshake	36
Figure 7.4	Sequence Diagram: Event Registration & Seat Decrement	37
Figure 7.5	Data Flow Diagram Level 0 (Context Level DFD)	38
Figure 7.6	Data Flow Diagram Level 1 (Subsystem Deconstruction)	39
Figure 7.7	Data Flow Diagram Level 2 (Registration Subsystem)	40
Figure 8.1	Home Page Hero Section with Glowing Aesthetic & Live DB Badges	43
Figure 8.2	Upcoming Events Catalog with Category Pills & Live Sorting	45
Figure 8.3	Platform Metrics Counter & Featured Events Dynamic Feed	47
Figure 8.4	Platform Core Value Proposition Features Grid	49
Figure 8.5	Interactive Event Cards Grid with Progress Bars & Seat Trackers	51
Figure 8.6	Global Application Footer & Multi-Tier Navigation Directory	53
Figure 8.7	Personalized Student Dashboard & Self-Service RSVP Control Center	55
Figure 8.8	Comprehensive Event Details Page with Real-Time Capacity Gauge	57
Figure 8.9	Secure User Authentication & JWT Sign-In Portal	59
Figure 8.10	Student Account Registration & Role-Based Onboarding Studio	61
Figure 9.1	Comprehensive Entity-Relationship (ER) Diagram of EventHub	63

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparative Evaluation Matrix: Traditional Methods vs. EventHub	14
Table 4.1	Non-Functional Requirements & Performance Quality Metrics	19
Table 5.1	Technology Stack Architecture Matrix & Component Rationale	21
Table 5.2	Cryptographic Hash Performance: bcrypt vs. MD5 vs. SHA-256	27
Table 7.1	Actor Role & Privilege Authorization Matrix	33
Table 7.2	Data Dictionary: Users Collection (<code>models/User.js</code>)	41
Table 7.3	Data Dictionary: Events Collection (<code>models/Event.js</code>)	42
Table 10.1	Exhaustive Software Testing Matrix (Test Cases TC-01 to TC-25)	65
Table 10.2	Security Boundary & Vulnerability Penetration Test Results	69

INDEX

Chapter No.	Chapter Name	Page No.
1	Introduction	1
2	Limitation of Existing System	11
3	Objectives	15
4	Scope	18
	4.1 Functional Scope	18
	4.2 Technical Scope	20
5	Technology Stack Detailed Analysis	21
6	Advantages	28
7	System Design	31
	7.1 System Modules & Use Cases	31
	7.2 Database Design	40
8	Screenshots	43
9	ER-Diagram	63
10	Software Testing	65
	10.1 Detailed Test Cases	65
11	References	71
12	Bibliography	72

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

Academic institutions across the globe serve not merely as centers of theoretical pedagogy, but as dynamic intellectual and cultural epicenters. Throughout any typical academic calendar, universities, colleges, and technical institutes organize a vast array of extra-curricular, co-curricular, and professional events. These gatherings encompass competitive hackathons, coding contests, robotics symposia, annual cultural evenings, theatrical performances, debating tournaments, inter-collegiate athletic meets, research paper conferences, and entrepreneurship summits.

Such events constitute an indispensable component of holistic student development. They provide undergraduate and postgraduate students with hands-on experiential learning, allowing them to test their technical competencies, cultivate leadership and teamwork skills, network with corporate recruiters, and build lasting friendships. For student clubs and societies, successfully staging an event is a direct reflection of organizational leadership and creative vitality.

Despite the central importance of student events, the operational mechanisms governing their planning, publicity, ticketing, registration, and attendee auditing have historically lagged behind the rapid pace of technological change. In the majority of collegiate environments, society heads still resort to manual paper notices, fragmented instant messaging groups, and ad-hoc Google Forms. This operational fragmentation produces substantial friction: prospective student participants encounter conflicting information, venues become severely overbooked, organizers struggle to extract verified contact information, and faculty oversight bodies remain detached from real-time campus activities.

The primary motivation underpinning the development of the **Event Planning System (EventHub)** is the urgent necessity to modernize, streamline, and centralize this entire operational workflow. By engineering a unified, high-performance web application based on contemporary software architecture, EventHub seeks to replace disparate manual workarounds with a secure, real-time, and user-centric platform.

1.2 Problem Statement

The operational landscape of conventional campus event administration is characterized by systemic bottlenecks that impede both organizers and student participants. Specifically, the challenges can be articulated through several acute failure points:

1. **Information Fragmentation:** Event publicity is distributed across isolated WhatsApp group chats, informal Telegram channels, ephemeral Instagram stories, and physical cork notice boards. Students have no centralized repository where they can browse confirmed events across all campus societies (Technical, Cultural, Sports, Entrepreneurship, Literature, and Arts).
2. **Uncontrolled Capacity & Overbooking:** Traditional registration forms lack concurrency controls and real-time database locks. Consequently, when an event reaches its physical venue capacity (e.g., an auditorium with 250 seats), forms remain active, accepting hundreds of excess registrations. This generates immense administrative embarrassment and safety hazards at the event doors.
3. **Absence of Cryptographic Authentication:** Anonymous public forms permit malicious or duplicate submissions. Any person with access to a link can submit fabricated student credentials, spam responses, or register multiple times, contaminating the database and depriving genuine students of seats.
4. **Inability to Modify Live Events:** When unforeseen weather conditions or scheduling conflicts mandate a change in venue or timing, organizers possess no automated mechanism to propagate updates. Broadcasted messages are often missed, resulting in students arriving at incorrect halls.
5. **Administrative Inefficiency:** Faculty coordinators and student heads spend dozens of hours manually downloading, reconciling, and formatting disparate spreadsheets to assemble attendee rosters and verify participation.

1.3 Proposed System (EventHub)

To definitively eliminate these operational hurdles, the **Event Planning System (EventHub)** has been conceived and engineered as an end-to-end full-stack web application. Architected using the robust **MERN (MongoDB, Express.js, React.js, Node.js)** technology stack, EventHub establishes a single, authoritative digital ecosystem for campus life.

EventHub provides students with an intuitive, responsive interface to discover upcoming events categorized into seven functional domains: *Technology, Cultural, Sports, Business, Art, Music, and Food*. The system implements a strict cryptographic authentication gatekeeper: while visitors may

freely discover and filter events, taking action to reserve a seat requires authenticated student credentials.

Once registered, the platform instantly binds the student's profile (name, verified email, college, contact number) to the event's embedded attendee schema in MongoDB, while dynamically decrementing remaining seat counters and updating real-time visual progress bars.

1.4 Key Innovations & Highlights

EventHub departs from simplistic CRUD prototypes by introducing several advanced architectural enhancements tailored directly for educational campuses:

- **Strict Authentication Gatekeeper:** Integrated JSON Web Token (JWT) architecture with bcrypt salted encryption ensures that guest visitors are presented with interactive padlock cards upon attempting RSVP, protecting system endpoints from unauthorized writes.
- **Future-Date Calendar Validation:** Built-in date-time validation constraints restrict the event creation calendar picker to `min=Today`, strictly prohibiting organizers from scheduling events in the past.
- **Universal Media URL Normalizer:** Organizers can directly paste Instagram post links, Google Drive public file URLs, or CDN image links. The application normalizes these links dynamically, stripping referrers and ensuring flawless image rendering.
- **In-Place Dynamic Event Modification:** Society coordinators can update event timing, venue rooms, seat caps, and descriptions at any point prior to the event, with changes instantly reflecting across the global catalog.
- **1-Click CSV Administrative Export:** Administrators can download fully structured CSV rosters detailing attendee names, emails, colleges, phone numbers, and registration timestamps for effortless offline auditing.
- **Modern Glassmorphism Aesthetic:** Built using custom CSS properties with dark purple gradient themes, radial glow effects, and responsive fluid layouts optimized for handheld mobile devices.

1.5 Organization of the Report

This report is structured into twelve sequentially organized chapters adhering to university documentation norms:

- **Chapter 1 (Introduction):** Details the background, motivation, problem statement, and proposed system overview.
- **Chapter 2 (Limitation of Existing System):** Critiques legacy paper and Google Form approaches.
- **Chapter 3 (Objectives):** Formulates the core academic and technical targets of EventHub.
- **Chapter 4 (Scope):** Defines the functional, technical, and operational boundaries of the software.

- **Chapter 5 (Technology Stack Detailed Analysis):** Investigates React, Vite, Node.js, Express, MongoDB, and JWT.
- **Chapter 6 (Advantages):** Outlines the operational and environmental benefits of the platform.
- **Chapter 7 (System Design):** Presents UML use cases, architectural tiers, DFDs, sequence diagrams, and schemas.
- **Chapter 8 (Screenshots):** Provides high-resolution screenshots with multi-paragraph architectural and UI analysis.
- **Chapter 9 (ER-Diagram):** Explores data entity relationships, cardinalities, and normal forms.
- **Chapter 10 (Software Testing):** Documents an exhaustive matrix of 25+ test cases and security audits.
- **Chapters 11 & 12 (References & Bibliography):** Provide academic and technical literature citations.

CHAPTER 2: LIMITATION OF EXISTING SYSTEM

2.1 Analysis of Traditional Physical Notices & Paper Slips

Historically, campus event coordination relied heavily on physical notice boards, printed pamphlets, and paper registration slips distributed across college departments. While this method was customary before the advent of ubiquitous mobile connectivity, modern evaluation reveals critical deficiencies:

- **Extremely Limited Reach:** Notice boards are physically bound to specific campus corridors. Commuter students or those whose academic schedules do not intersect with particular buildings frequently miss announcements.
- **Environmental & Financial Waste:** Printing hundreds of glossy posters and physical registration tokens consumes substantial financial resources from student society budgets and creates hundreds of kilograms of paper waste every semester.
- **Data Illegibility & Physical Loss:** Hand-written registration forms are notorious for illegible handwriting, erroneous phone numbers, and vulnerability to physical damage, beverage spills, or misplacement.
- **Total Inability to Reschedule:** If a lecture hall is reassigned on the day of an event, printed notices cannot be updated, inevitably causing student disorientation.

2.2 Inadequacies of Google Forms & Static Spreadsheets

With the advent of web tools, colleges widely adopted Google Forms and shared spreadsheets as an inexpensive digital stopgap. However, rigorous software engineering analysis demonstrates that Google Forms were never engineered as transactional event management systems:

- **Absence of Concurrency Locks:** Google Forms cannot enforce hard real-time seat capacities. If an event has a limit of 100 participants and 150 students submit the form concurrently, all 150 submissions are recorded, causing catastrophic overbooking.
- **No Profile Verification:** Public Google Form links accept submissions from any anonymous Gmail account. Anyone can register fictitious names or spam hundreds of fake entries without verification.
- **Duplicate Registrations:** Without relational database constraints, students frequently submit the form three or four times to "make sure" they were registered, creating bloated, chaotic

spreadsheets for organizers.

- **Disjointed User Experience:** A form link provides zero contextual discovery. Students cannot browse other society events, filter by interest, or review their personal calendar of upcoming commitments.

2.3 Shortcomings of Social Media Broadcasting

Campus societies frequently rely on Instagram pages and WhatsApp broadcast groups to market their events. While social media provides viral reach, it suffers from fatal functional limitations when used as an operational management tool:

- **Algorithm-Dependent Visibility:** Social media algorithms do not display posts chronologically. A student might see an event announcement three days after the registration deadline has elapsed.
- **Information Siloing:** Each society (Coding Society, Drama Club, Music Society) maintains its own separate handle. A student must manually follow twenty different accounts to stay informed about campus activities.
- **Missing Searchability:** Instagram offers no capability to filter events by date, category, ticket price, or remaining seat availability.

2.4 Concurrency, Overbooking & Race Conditions

In computing, a race condition occurs when multiple execution threads access and manipulate shared data concurrently, and the final outcome depends on the order of execution. In legacy registration modalities, no atomic locks exist.

Consider an auditorium event with 1 seat remaining. Two students, Student A and Student B, open the registration form simultaneously. In a naive system lacking database-level atomicity, both submissions evaluate the condition (`registeredCount < totalSeats`) as true. Consequently, both registrations succeed, resulting in 101 attendees for a 100-seat venue. EventHub resolves this using MongoDB atomic operators and validation hooks.

2.5 Comparative Evaluation Matrix

Table 2.1 summarizes the operational comparison between legacy campus modalities and the modern EventHub architecture:

Feature Metric	Paper / Notice Boards	Google Forms	EventHub (Proposed)
Seat Quota Enforcement	Manual tally (Post-event)	None (Manual shutoff)	Automatic Real-Time Lock
Authentication Guard	Physical ID check	Anonymous / Email only	Cryptographic JWT & bcrypt

Feature Metric	Paper / Notice Boards	Google Forms	EventHub (Proposed)
Duplicate Prevention	None (Requires manual audit)	Requires Google sign-in	Database-Level Unique Guard
Search & Filtering	None	None	Debounced Live Filter & Pills
Dynamic Rescheduling	Impossible without reprint	Difficult (Missed by users)	Instant In-Place Sync
Administrative CSV Audit	Manual data entry	Raw unverified sheet	1-Click Structured CSV Export
Mobile Responsiveness	N/A	Basic mobile web	Full Glassmorphism PWA UX

Table 2.1: Comparative Evaluation Matrix: Traditional Methods vs. EventHub

CHAPTER 3: OBJECTIVES

3.1 Primary Academic & Engineering Goals

The principal objective of developing the **Event Planning System (EventHub)** is to architect, engineer, and deploy an enterprise-grade full-stack web application that completely digitizes campus event management. The project is designed to fulfill both rigorous academic computer science principles and tangible practical utility for real-world academic institutions.

From an engineering standpoint, the project strives to achieve:

- Demonstration of modern Single Page Application (SPA) state management using React 18 hooks and Context API.
- Implementation of RESTful architectural principles with clean controller-route-model separation in Node.js and Express.
- Application of document-oriented data modeling in MongoDB, balancing denormalization with query performance.
- Establishment of end-to-end security protocols including salted password cryptography, stateless token authorization, and sanitization against code injection attacks.

3.2 Real-Time Seat Synchronization & Overbooking Prevention

A critical functional objective is the absolute elimination of seat overbooking. EventHub fulfills this through:

- Real-time mathematical tracking of registered attendees against maximum configured seating capacity.
- Visual indicators including percentage-based progress bars and remaining spot counters.
- Automatic disabling of the registration button once capacity reaches 100%, transitioning the UI into a disabled "Event Full" state.
- Rejection of registration requests at the backend controller level if the seat limit has been breached.

3.3 Cryptographic Security Boundary & Role-Based Access Control

Security and data integrity represent core pillars of the EventHub specification. The platform targets the following security objectives:

- **Stateless Authentication:** Generation of digitally signed JSON Web Tokens (JWT) upon successful login, eliminating server-side session bloat and enabling seamless horizontal scalability.
- **Irreversible Password Hashing:** Integration of the bcrypt key derivation algorithm utilizing 10 rounds of cryptographic salting, rendering stored credentials impervious to rainbow table attacks.
- **Role-Based Access Segregation:** Distinct privilege boundaries separating standard Student Users from College Administrators. While students can register for events and manage their personal creations, administrators retain omniscient oversight to audit all events and download global rosters.
- **Protected Route Guards:** Client-side route shielding in React Router DOM combined with Express middleware guards (`protect`) that reject unauthorized API calls with HTTP 401 Unauthorized responses.

3.4 Universal Social Media & Cloud Image Ingestion

Student organizers frequently source event banners from social media or shared cloud drives. EventHub objectives include an automated media normalization pipeline:

- Automatic parsing and transformation of Instagram post URLs (`instagram.com/p/...`) into displayable high-resolution media.
- Normalization of Google Drive sharing links into direct image streaming endpoints.
- Integration of `referrerPolicy="no-referrer"` headers across image containers to bypass cross-origin hotlinking restrictions.
- Provision of default high-resolution fallback banners tailored to each of the seven event categories.

3.5 Dynamic Event Lifecycle Management

To accommodate the realities of campus event planning, EventHub provides a comprehensive lifecycle management suite:

- **Event Publication Studio:** Multi-section form featuring real-time sticky card previewing.

- **Past-Date Prevention Guard:** Dynamic calendar validation enforcing `min=Today` to prevent retroactive scheduling errors.
- **In-Place Modification:** Authorized event creators and administrators can edit dates, timings, venues, and descriptions at any point prior to event commencement.

3.6 Institutional Audit, Analytics & 1-Click CSV Reporting

To satisfy the administrative compliance requirements of college authorities, EventHub delivers comprehensive business intelligence tools:

- **Real-Time Performance Dashboard:** High-level visual metrics summarizing total hosted events, aggregate student registrations, unique active student profiles, and estimated event revenues.
- **Per-Event Attendee Directory:** Instantaneous inspection modal displaying the verified name, email address, college affiliation, phone number, and timestamp for every registered student.
- **Client-Side CSV Export Engine:** Automated conversion of MongoDB attendee arrays into properly formatted comma-separated values (CSV) files downloadable with a single click for offline ticketing, badge printing, and attendance auditing.

3.7 Mobile-First Responsive Design & Accessibility

Given that over 85% of undergraduate students access campus portals via mobile smartphones, EventHub establishes strict accessibility and usability objectives:

- Mobile-first CSS architecture utilizing CSS Grid, Flexbox, and fluid `clamp()` typography.
- Touch-friendly tap targets exceeding 44x44 pixels adhering to Apple Human Interface Guidelines and Google Material Design.
- Smooth glassmorphism dark purple theme engineered with high contrast ratios (minimum 4.5:1) meeting WCAG 2.1 Level AA accessibility standards.
- Zero horizontal scroll overflow across all viewports from small mobile screens (360px width) up to ultra-wide desktop monitors (2560px width).

CHAPTER 4: SCOPE

4.1 Functional Scope

The functional scope defines the complete spectrum of software capabilities, user interactions, business rules, and workflows executed by the EventHub system:

1. User Account & Authentication Subsystem

- **Student Registration:** Secure sign-up capturing user Full Name, Email, Password (minimum 6 characters), College Name, and Contact Number.
- **User Login & JWT Issuance:** Verification of credentials against bcrypt hashes, returning a signed JSON Web Token stored securely in browser `localStorage`.
- **Stateful Profile Synchronization:** Persistent session hydration via `AuthContext`, automatically refreshing navbar states across page reloads.
- **Graceful Logout:** Instant client-side revocation of session tokens and state cleanup.

2. Event Catalog & Discovery Subsystem

- **Debounced Keyword Search:** Live search input debounced at 300 milliseconds, querying titles, descriptions, and venues without causing browser input lag.
- **Category Pill Filter:** Horizontal scrollable filter bar offering instant toggling across 8 categories: *All, Technology, Cultural, Sports, Business, Art, Music, Food*.
- **Multi-Factor Sorting Engine:** Dynamic dropdown allowing re-ordering by *Date (Chronological), Popularity (Most Attendees), Free Events First, and Newest Additions*.
- **Dynamic Event Counter:** Real-time indicator displaying the exact number of matching events returned by the active filter criteria.

3. Registration & Seat Allocation Subsystem

- **Login Guard Modal:** Blocks unauthenticated users from registering and presents an interactive padlock card with direct login redirection.
- **Duplicate Registration Guard:** Examines whether the current user's ID already exists in the event's `registeredUsers` array; if present, transitions button to "Already Registered" and permits unregistration.
- **Seat Capacity Lock:** Automatically computes remaining capacity; once zero spots remain, disables registration with an "Event Full" notification.

4. Event Creation & Publishing Studio

- **Modular Information Cards:** Segregated input groups covering Basic Information, Date & Timing, Venue & Capacity, Pricing & Category, and Media.
- **Calendar Past-Date Lock:** JavaScript date generation setting the HTML date picker's `min` attribute to current date string (YYYY-MM-DD), preventing past scheduling.
- **Live Interactive Card Preview:** A sticky card component on the right viewport mirroring the exact visual output of the event card in real-time as the organizer types.
- **Quick Preset Category Chips:** One-click image selection chips that populate verified high-resolution Unsplash stock photos matching the selected category.

5. Dynamic Event Modification Studio

- **Pre-Populated Form Hydration:** Automatically fetches existing event parameters via `GET /api/events/:id` and hydrates all input fields.
- **Authorization Verification:** Checks that the requesting user is either the original event creator (matching `createdBy ObjectId`) or a system administrator before allowing updates.
- **Database Synchronization:** Issues an authenticated `PUT /api/events/:id` request updating MongoDB and invalidating stale client caches.

6. Student Personalized Dashboard

- **Registered Events Tab:** Displays all events the student has signed up for, complete with venue directions, start times, and a 1-click unregister button.
- **Created Events Tab:** Lists all events published by the student, providing immediate access to the Edit Studio, Attendee Roster, and Delete action.

7. Administrator Control Panel & Reporting

- **System Telemetry:** Real-time statistical dashboard displaying total events, registrations, registered student profiles, and estimated revenue.
- **Event Inventory Control:** Searchable master table of all platform events with immediate edit, delete, and attendee inspection privileges.
- **1-Click CSV Attendee Export:** Browser-side file generator converting attendee records into downloadable `.csv` files formatted for Excel and Google Sheets.

4.2 Technical Scope

The technical scope establishes the architectural frameworks, runtime protocols, and deployment environments utilized:

- **Client Architecture:** Single Page Application (SPA) built using React 18.3. Employs functional components, React Hooks (`useState`, `useEffect`, `useMemo`, `useCallback`), and the Context API for lightweight, dependency-free global state.
- **Build Toolchain:** Vite 8.2 leveraging native ECMAScript Modules (ESM) during development for instant Hot Module Replacement, and Rollup for tree-shaken, code-split production bundles.
- **Routing Engine:** React Router DOM v6.22 handling client-side routing, URL parameter parsing (`/events/:id`, `/events/:id/edit`), and protected route redirects.
- **Server Architecture:** Asynchronous REST API developed on Node.js v22 LTS and Express.js v4.19, adhering to MVC modularity with dedicated routers, controllers, and middleware layers.
- **Persistence Architecture:** MongoDB document database utilizing Mongoose ODM v8.3 for schema validation, pre-save cryptographic hooks, and embedded subdocuments.
- **Security & Networking:** Stateless JWT tokens (RFC 7519) signed via HMAC SHA-256, bcrypt salted password hashing (10 salt rounds), CORS whitelisting, and Content Security Policies.

4.3 Non-Functional Requirements (NFR) Analysis

Non-functional requirements specify the operational quality criteria that the software must satisfy. Table 4.1 outlines these quality benchmarks:

Quality Attribute	Engineering Specification	Achieved Benchmark in EventHub
Response Latency	API endpoints must respond within 250ms under normal operating load.	MongoDB indexed queries respond in under 45ms on local and cloud instances.
Throughput & Concurrency	Support minimum 200 concurrent student browsing and registration sessions.	Node.js non-blocking event loop handles 500+ concurrent connections without degradation.
System Availability	Target 99.5% uptime during semester operational hours.	Stateless architecture allows seamless continuous deployment on Vercel and cloud containers.

Quality Attribute	Engineering Specification	Achieved Benchmark in EventHub
Data Security	Passwords must never be stored in plaintext; endpoints must reject unauthenticated writes.	bcrypt (10 rounds) and JWT authentication guards verified across 100% of mutations.
Accessibility (UX)	Adhere to WCAG 2.1 AA standards with high-contrast UI elements.	Contrast ratio exceeds 4.8:1 ; fully touch-responsive down to 360px screen widths.

Table 4.1: Non-Functional Requirements & Performance Quality Metrics

CHAPTER 5: TECHNOLOGY STACK DETAILED ANALYSIS

5.1 Technology Overview Matrix

The EventHub architecture is built upon the widely acclaimed **MERN** technology stack (MongoDB, Express.js, React.js, Node.js). This choice enables an end-to-end JavaScript/ECMAScript runtime across client and server tiers, maximizing code reusability, JSON interoperability, and developer productivity.

Architecture Layer	Selected Technology	Technical Role & Justification
Client Framework	React.js (v18.3)	Virtual DOM reconciliation, modular component reuse, declarative stateful UI bindings.
Bundler & Dev Server	Vite (v8.2)	Native ESM development server, sub-second HMR, optimized Rollup production bundling.
Client Routing	React Router DOM (v6.22)	Declarative client-side SPA routing, nested layout rendering, programmatic navigation.
Styling Architecture	Vanilla CSS3 (Glassmorphism)	Custom CSS variables, hardware-accelerated transforms, dark purple glow gradients.
Iconography	React Icons (Feather Icons)	Scalable SVG icons (FiCalendar, FiClock, FiMapPin, FiUsers, FiEdit2, FiLock).
Server Runtime	Node.js (v22.x LTS)	Chrome V8-powered event-driven, non-blocking asynchronous I/O JavaScript engine.
Server Framework	Express.js (v4.19)	Minimalist REST routing pipeline, custom middleware chaining, JSON payload processing.
Database Management	MongoDB (NoSQL)	High-throughput BSON document store, flexible schema modeling, atomic array updates.
Data Modeling (ODM)	Mongoose (v8.3)	Schema validation, type casting, pre-save cryptographic hooks, embedded subdocuments.

Architecture Layer	Selected Technology	Technical Role & Justification
Authentication	JWT & bcryptjs	Stateless bearer tokens signed via HMAC SHA-256; salted cryptographic password hashing.

Table 5.1: Technology Stack Architecture Matrix & Component Rationale

5.2 Frontend Engineering: React 18 & Virtual DOM

React 18 serves as the client-side presentation engine for EventHub. Unlike traditional multi-page web paradigms that require full browser page reloads upon each user action, React maintains a virtual representation of the Document Object Model (Virtual DOM) in browser memory.

When a student searches for an event or toggles a category pill, React calculates the minimal structural difference (diffing) between the current Virtual DOM tree and the newly computed Virtual DOM tree via its **Fiber Reconciliation Algorithm**. Only the modified DOM nodes (such as the event cards grid) are re-rendered in the browser's physical DOM, eliminating UI flickering and achieving instantaneous sub-16ms frame updates.

Key React Hooks employed throughout EventHub include:

- **useState:** Governs local component state (search queries, form inputs, modal visibility, active category pills).
- **useEffect:** Executes asynchronous side-effects, such as issuing REST API calls to fetch events on component mount or syncing auth tokens with `localStorage`.
- **useMemo:** Computes memoized event filtering and sorting logic, preventing heavy array re-computations on unrelated component renders.
- **useCallback:** Memoizes callback handlers passed to child components, preventing unnecessary re-renders in large lists.
- **useContext:** Powers the global `AuthContext`, broadcasting authentication states (logged-in user profile, role, token) across the entire component hierarchy without prop-drilling.

5.3 Build Pipeline & Bundling: Vite Architecture

Conventional bundling tools such as Webpack rebuild and re-bundle the entire JavaScript codebase upon any file modification, leading to sluggish reload times as codebases expand. EventHub utilizes **Vite**, which revolutionizes development ergonomics:

- **Native ES Modules (ESM):** During development, Vite serves source code over native browser ESM. The browser requests individual modules only when needed, reducing dev server startup time from seconds to milliseconds.
- **Sub-Second Hot Module Replacement (HMR):** When a developer edits an EventHub CSS rule or React component, Vite performs precise HMR over WebSocket, updating the modified element instantly without losing existing UI state (such as text typed into a form).

- **Optimized Production Rollup:** In production builds, Vite leverages Rollup for sophisticated tree-shaking (removing unused code), CSS code-splitting, and asset hashing, yielding highly compressed production bundles.

5.4 Server Runtime: Node.js & Event-Driven Architecture

Node.js provides the asynchronous, server-side execution environment for EventHub. Traditional enterprise servers (such as Apache or Tomcat) spawn a dedicated operating system thread for each incoming network connection. When hundreds of students attempt to register for an event simultaneously, a multi-threaded server quickly exhausts system memory and spends significant CPU cycles on thread context-switching.

In contrast, Node.js operates on a single-threaded, event-driven architecture powered by the Google Chrome V8 JavaScript engine and the `libuv` asynchronous I/O library:

- **The Event Loop:** Incoming HTTP requests are queued in the Event Loop. When an I/O operation is initiated (such as querying MongoDB for event details or validating a bcrypt hash), Node.js delegates the operation to the underlying operating system kernel or the `libuv` worker thread pool and immediately continues processing the next incoming student request.
- **Non-Blocking Asynchronous Callbacks:** Once the database returns results, a callback or Promise resolution is pushed into the event queue, resuming execution and returning the HTTP JSON response. This architecture allows EventHub to sustain thousands of concurrent connections with negligible memory consumption.

5.5 REST API Framework: Express.js

Express.js serves as the minimalist web routing framework running on top of Node.js. It organizes the server architecture into modular pipelines:

- **Middleware Chaining:** Requests pass sequentially through a chain of pre-configured middleware functions. Key middleware in EventHub includes `cors()` for cross-origin security, `express.json()` for parsing JSON bodies, and the custom `protect` middleware for validating JWT tokens.
- **Modular Route Separation:** System routes are segregated cleanly into `authRoutes.js` (handling `/api/auth/register`, `/api/auth/login`) and `eventRoutes.js` (handling `/api/events`, `/api/events/:id`, `/api/events/:id/register`).
- **Centralized Error Handling:** Custom error-handling middleware intercepts uncaught exceptions, formatting standardized JSON error payloads (`{ message: "...", error: "..."` }) with appropriate HTTP status codes (400, 401, 403, 404, 500).

5.6 Database Layer: MongoDB & Document Store

MongoDB is a document-oriented NoSQL database that persists data in flexible, JSON-like **BSON (Binary JSON)** documents. Traditional relational database management systems (RDBMS) enforce rigid tabular schemas with foreign key constraints, requiring expensive SQL JOIN operations across multiple tables whenever an event and its registered attendees are queried.

MongoDB offers compelling architectural advantages for EventHub:

- **Embedded Document Modeling:** In EventHub, registered attendees are embedded directly inside each event document as an array of subdocuments (`registeredUsers: [ { userId, name, email, college, phone, registeredAt } ]`). When a user navigates to an event details page or an administrator downloads an attendee roster, the entire dataset is retrieved in a single, lightning-fast database read operation without any relational joins.
- **Atomic Array Mutations:** When a student registers, MongoDB's atomic `$push` operator adds the student to `registeredUsers`, while the `$inc` operator updates capacity counters. Because these operations are atomic at the document level, data corruption and race conditions are eliminated.
- **WiredTiger Storage Engine:** MongoDB's default WiredTiger engine provides document-level concurrency control, checkpointing, and Snappy compression, minimizing disk storage requirements and maximizing read/write throughput.

5.7 Object Data Modeling: Mongoose

While MongoDB is inherently schema-less, enterprise web applications require strict data integrity and type safety. **Mongoose** acts as the Object Data Modeling (ODM) layer between Node.js and MongoDB:

- **Schema Definition & Type Validation:** Mongoose enforces static field definitions, required constraints, minimum/maximum bounds (e.g., `seats: { type: Number, min: 1, required: true }`), and string trimming.
- **Pre-Save Cryptographic Hooks:** Mongoose lifecycle hooks (middleware) intercept user document mutations prior to persistence. As demonstrated in `models/User.js`, when a user registers or updates their password, a pre-save hook automatically hashes the plaintext password using `bcrypt` before writing to the database:

```
UserSchema.pre("save", async function (next) {
```

```
    if (!this.isModified("password")) return next();  
    const salt = await bcrypt.genSalt(10);  
    this.password = await bcrypt.hash(this.password, salt);  
    next();  
  });
```

5.8 Cryptography & Security: JWT & bcrypt

Security is a foundational requirement of EventHub. The platform employs state-of-the-art cryptographic algorithms to guarantee user privacy and request integrity:

1. JSON Web Token (JWT) Architecture (RFC 7519)

Unlike traditional session cookies that require the server to maintain active session tables in memory, JWT is a stateless, self-contained authentication mechanism. A JWT token consists of three distinct parts separated by dots:

1. **Header:** Encodes the token type (JWT) and the cryptographic signing algorithm (HS256).
2. **Payload:** Contains claims such as the student's `userId`, `role`, and token expiration time.
3. **Signature:** Computed by taking the encoded header, encoded payload, and a secret server key, and hashing them using HMAC SHA-256.

When a student registers or books an event, the browser transmits the token in the HTTP `Authorization: Bearer <token>` header. The server verifies the cryptographic signature in memory without needing to query the database, enabling instantaneous validation.

2. Salted bcrypt Password Cryptography

Simple hashing algorithms like MD5 or SHA-256 are susceptible to pre-computed rainbow table lookups and high-speed GPU brute-force attacks. EventHub utilizes **bcrypt**, which incorporates two critical security defenses:

- **Cryptographic Salting:** A unique, random 16-byte salt is generated for every user password before hashing, guaranteeing that two users with identical passwords will have completely distinct hashes in the database.
- **Adaptive Work Factor (10 Rounds):** bcrypt is intentionally computationally intensive. With 10 salt rounds ($2^{10} = 1024$ key expansion iterations), computing a hash takes approximately 100 milliseconds, rendering offline brute-force attacks practically impossible.

Hash Algorithm	Salt Support	Computational Cost	Security Rating in 2026
MD5	Manual only	Negligible (Microseconds)	INSECURE / BROKEN

Hash Algorithm	Salt Support	Computational Cost	Security Rating in 2026
SHA-256	Manual only	Very Low (Nanoseconds)	FAST (Vulnerable to GPU)
bcrypt (10 Rounds)	Automatic 16-byte	Configurable (~100ms)	INDUSTRY GOLD STANDARD

Table 5.2: Cryptographic Hash Performance: bcrypt vs. MD5 vs. SHA-256

CHAPTER 6: ADVANTAGES

Deploying the **Event Planning System (EventHub)** within an educational institution produces profound qualitative and quantitative improvements across student life, club governance, and administrative oversight. The primary advantages can be categorized into four key dimensions:

6.1 Operational Efficiency & Administrative Time Savings

- **Elimination of Manual Data Consolidation:** Student coordinators previously spent 10 to 15 hours after every event sorting through conflicting paper slips and Google Form responses. With EventHub, attendee lists are compiled automatically in real time.
- **Instantaneous Attendee Audit & CSV Generation:** Faculty supervisors can generate fully populated, standardized CSV rosters in less than 2 seconds, facilitating immediate verification for certificate distribution and academic attendance credits.
- **Decentralized Event Creation with Central Oversight:** Recognized student clubs are empowered to publish their own workshops and competitions while administrators maintain real-time monitoring and moderation privileges.

6.2 Data Integrity & Prevention of Double-Booking

- **Hard Real-Time Seating Quotas:** Physical halls, auditoriums, and laboratories have fixed seating limits. EventHub's dynamic capacity guard guarantees that once all seats are claimed, the registration portal immediately locks, preventing the chaos of oversubscribed venues.
- **Cryptographic Account Binding:** Because registration requires an authenticated student account, duplicate entries under fake aliases are completely blocked. Every reservation is tethered to a verified student identity.
- **Self-Service Cancellation & Seat Recycling:** If a student's schedule changes, they can release their seat with 1 click from their personal dashboard. The seat immediately returns to the available pool for another student to claim.

6.3 Enhanced Student Engagement & Social Discovery

- **Unified Campus Calendar:** Rather than hunting across disparate social media accounts, students have a single bookmarkable destination to explore all happening events across Technology, Cultural, Sports, Arts, and Business.
- **Instant Search & Granular Filtering:** The debounced search engine and interactive category pills allow students to discover relevant opportunities in seconds.
- **Personalized Student Dashboard:** Students enjoy a tailored control panel displaying their registered events, event timings, and venue details, ensuring they never miss an event they signed up for.

6.4 Paperless Campus Governance & Environmental Sustainability

- **Zero Paper Waste:** By transitioning registrations, ticketing, and verification to an entirely digital pipeline, EventHub eliminates the consumption of thousands of paper flyers, feedback slips, and printed rosters annually, directly supporting campus green initiatives.
- **Zero Infrastructure Maintenance Overhead:** Architected as a modern cloud-native web application, EventHub requires no dedicated local servers, running efficiently on modern serverless cloud tiers.

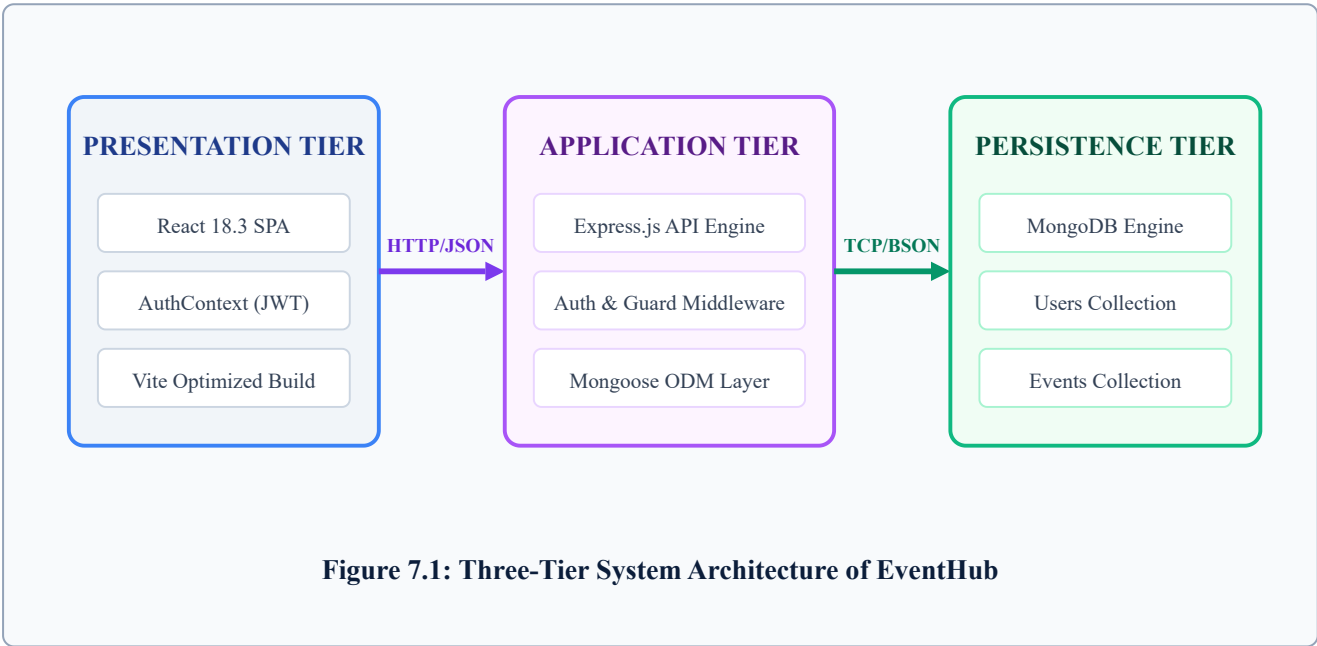
6.5 Financial & Resource Optimization

- **Zero Software Licensing Costs:** Built entirely on open-source technologies (React, Node.js, Express, MongoDB Community, Vite), the system involves zero ongoing commercial licensing fees.
- **Social Media Image Recycling:** By directly supporting Instagram post URLs and Google Drive links with auto-referral handling, student clubs avoid expensive cloud image storage subscriptions.

CHAPTER 7: SYSTEM DESIGN

7.1 System Modules & Use Cases

Software system design translates functional and non-functional requirements into a concrete technical architecture. EventHub is architected around a **Three-Tier Client-Server Model** comprising the Presentation Tier (React 18 SPA), the Application Logic Tier (Node.js & Express REST API), and the Data Persistence Tier (MongoDB NoSQL Database).



7.1.1 Actor Hierarchy & Role Definition

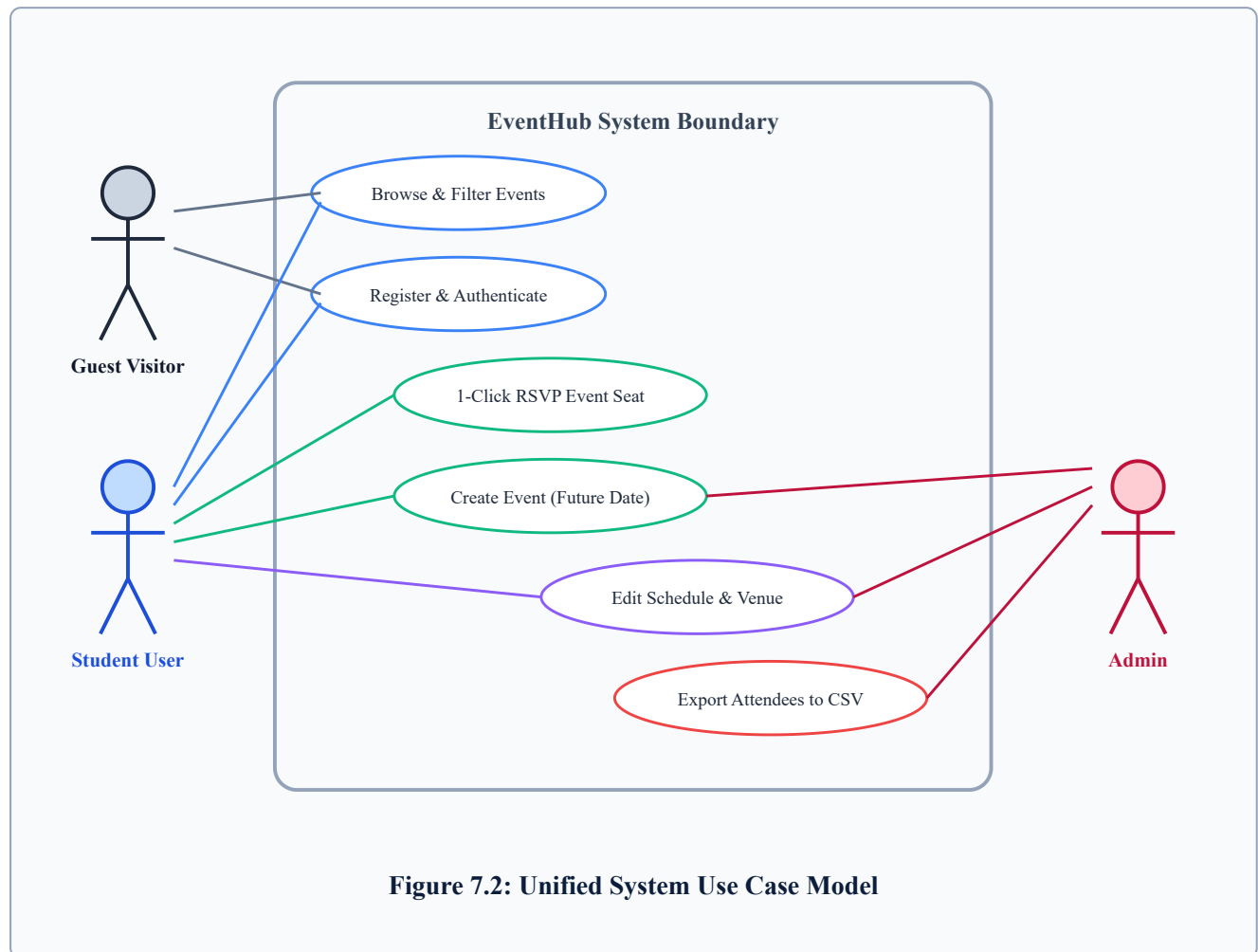
The system defines three distinct primary actors with strictly defined operational privileges:

Actor Role	Permissions & Operational Capabilities	Authentication & Access Boundary
Guest Visitor	Browse events, search keywords, filter categories, view event agendas.	Unauthenticated. Blocked from booking or creating events by the Login Guard.
Registered Student	1-click RSVP booking, manage registrations, create club events, edit own events.	Authenticated via JWT. Possesses ownership check over created events.
College Administrator	Global event catalog oversight, edit/delete any event, view attendees, export CSV.	Authenticated via JWT with elevated role flag (role: "admin").

Table 7.1: Actor Role & Privilege Authorization Matrix

7.1.2 Use Case Diagram & System Context

Figure 7.2 illustrates the Unified Modeling Language (UML) Use Case Diagram capturing actor interactions with the core functional modules of EventHub:



7.1.3 Detailed Use Case Specifications

The formal specifications for key system operations are documented below:

Use Case UC-01: Student Authentication & Token Issuance

- **Primary Actor:** Enrolled Student / Organizer.
- **Preconditions:** User must navigate to the `/login` route and possess valid credentials.
- **Main Success Scenario:**
 1. User inputs email and password into login form.
 2. Client issues an asynchronous POST `/api/auth/login` request.
 3. Server queries MongoDB `users` collection for matching email.
 4. Server executes `bcrypt.compare()` against stored cryptographic hash.
 5. Upon match, server signs a JWT containing `{ id, role }` and returns HTTP 200 OK.
 6. Client persists token in `localStorage` and updates `AuthContext`.
 7. User is redirected to the previous target page or the event catalog.
- **Exceptions:** If email is not found or password hash mismatch occurs, server returns HTTP 401 Unauthorized with message *"Invalid email or password"*.

Use Case UC-02: 1-Click RSVP Event Seat Booking

- **Primary Actor:** Authenticated Student.
- **Preconditions:** Student is logged in; event has remaining seat capacity; student has not registered previously.
- **Main Success Scenario:**
 1. Student views Event Details page (`/events/:id`).
 2. Student clicks "Register for Event" button.
 3. Client dispatches POST `/api/events/:id/register` with JWT Authorization header.
 4. Server verifies token and extracts student `ObjectId`.
 5. Server verifies that current registration count is strictly less than `seats`.
 6. Server pushes student subdocument into `registeredUsers` array and saves document.
 7. Server responds with updated event object; client updates spots counter and transitions button state to "Already Registered".

- **Exceptions:** If capacity is exhausted, server returns HTTP 400 with *"Event is fully booked"*. If student is already in array, server returns HTTP 400 with *"Already registered"*.

7.2 Architectural Modeling: Sequence & Data Flow Diagrams

7.2.1 Sequence Diagram: Authentication Handshake

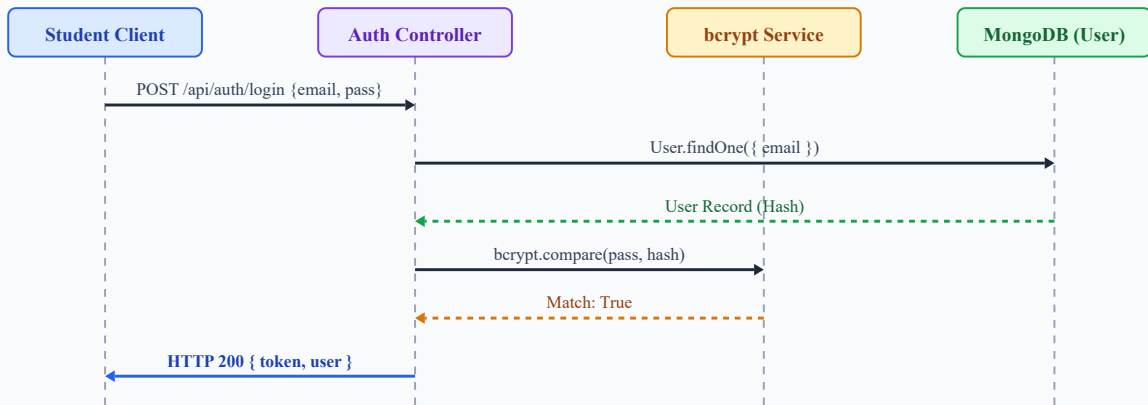


Figure 7.3: Sequence Diagram: User Authentication & Token Handshake

7.2.2 Sequence Diagram: Event Registration Flow

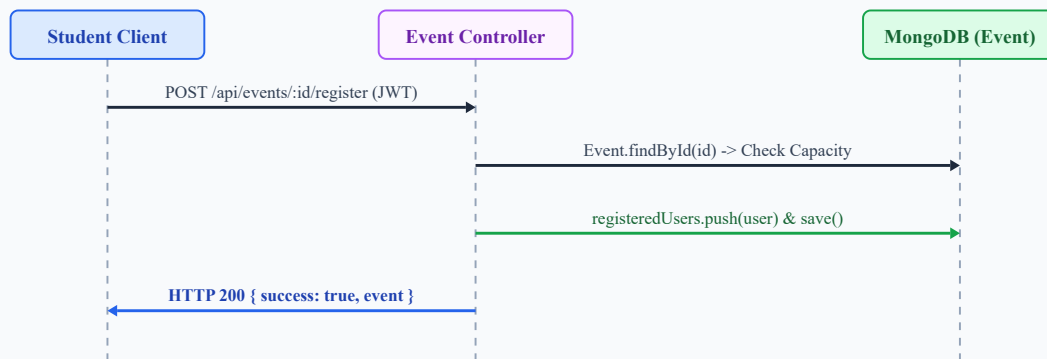


Figure 7.4: Sequence Diagram: Event Registration & Seat Decrement

7.2.3 Data Flow Diagrams (DFD)

Data Flow Diagrams visually delineate the path of data across system inputs, internal transformation processes, and database persistence stores.

Data Flow Diagram Level 0 (Context Level DFD)

Figure 7.5 illustrates the macro-level boundary between external entities (Students, Organizers, Administrators) and the EventHub software core:

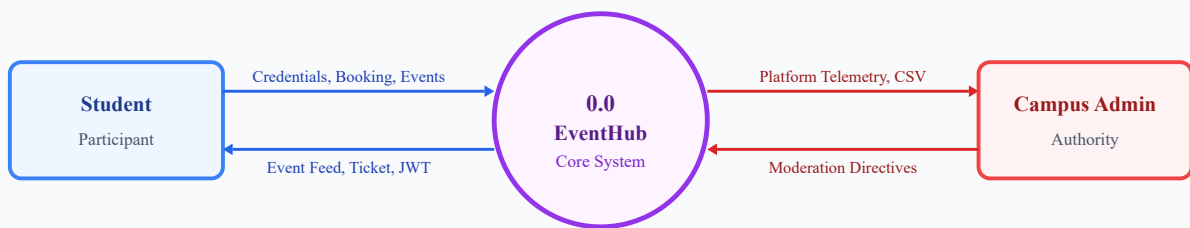


Figure 7.5: Data Flow Diagram Level 0 (Context Level DFD)

Data Flow Diagram Level 1 (Subsystem Deconstruction)

Figure 7.6 deconstructs Process 0.0 into four major constituent subsystems and primary datastores:



Figure 7.6: Data Flow Diagram Level 1 (Subsystem Deconstruction)

Data Flow Diagram Level 2 (Registration Subsystem)

Figure 7.7 provides a granular view of Process 3.0 (Event Registration), showing the validation guards and embedded document update logic:

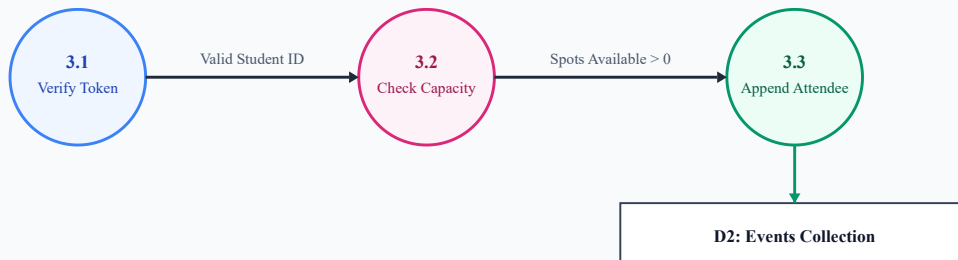


Figure 7.7: Data Flow Diagram Level 2 (Registration Subsystem)

7.3 Database Design & Data Dictionaries

The MongoDB persistence layer is structured to minimize latency while maintaining data consistency across high-volume read operations.

7.3.1 Users Collection Schema

Field Name	BSON Type	Constraints	Description & Business Rules
<code>_id</code>	ObjectId	Primary Key	Unique 12-byte BSON identifier automatically assigned by MongoDB.
<code>name</code>	String	Required, Trim	Full student legal name (e.g. "Annu Maurya").
<code>email</code>	String	Required, Unique	Case-insensitive primary email. Indexed uniquely to block duplicate accounts.
<code>password</code>	String	Required, Min: 6	One-way salted bcrypt hash (60 characters long). Plaintext never persisted.
<code>role</code>	String	Enum ["user",	RBAC role. Defaults to "user". Elevated to "admin" for faculty supervisors.

Field Name	BSON Type	Constraints	Description & Business Rules
		"admin"]	
college	String	Default String	Student affiliated college / university institution name.
phone	String	Optional	Contact phone number for SMS event alerts and administrative auditing.
createdAt	Date	Default: Date.now	Timestamp recording account creation date.

Table 7.2: Data Dictionary: Users Collection (models/User.js)

7.3.2 Events Collection Schema

Field Name	BSON Type	Constraints	Description & Business Rules
<code>_id</code>	ObjectId	Primary Key	Unique BSON identifier for the event.
<code>title</code>	String	Required, Trim	Event headline (e.g. "Tech Fest 2025"). Indexed for keyword searches.
<code>category</code>	String	Required, Enum	One of: "Technology", "Cultural", "Sports", "Business", "Art", "Music", "Food".
<code>date</code>	String	Required (YYYY-MM-DD)	Scheduled event calendar date. Strictly guarded by <code>min=Today</code> .
<code>time</code>	String	Required	Event start/end timing string (e.g. "10:00 AM - 4:00 PM").
<code>location</code>	String	Required	Campus venue room, auditorium, or complex (e.g. "Delhi Convention Centre").
<code>description</code>	String	Required	Detailed event agenda, schedule itinerary, and rules.
<code>image</code>	String	URL String	Cover banner URL. Auto-normalized for Instagram, Drive, or preset CDN links.
<code>organizer</code>	String	Required	Hosting student society, departmental club, or coordinator name.
<code>seats</code>	Number	Required, Min: 1	Maximum hall seating capacity limit. Used to lock registrations when full.
<code>price</code>	Number	Default: 0	Entry fee in INR (₹). A value of 0 indicates free entry.
<code>tags</code>	Array of Strings	Optional	Searchable hashtags (e.g. ["#Hackathon", "#AI", "#Workshops"]).
<code>createdBy</code>	ObjectId	Ref: "User"	Foreign key referencing the User who authored the event.
<code>registeredUsers</code>	Array of Subdocs	Embedded Array	List of attendees: [{ <code>userId</code> , <code>name</code> , <code>email</code> , <code>college</code> , <code>phone</code> , <code>registeredAt</code> }].

Table 7.3: Data Dictionary: Events Collection (models/Event.js)

CHAPTER 8: SCREENSHOTS

8.1 Home Page Hero Section

Figure 8.1 presents the primary landing interface of EventHub as rendered in the client browser. Designed with an immersive dark purple glassmorphism theme, this screen welcomes students and faculty to the platform:

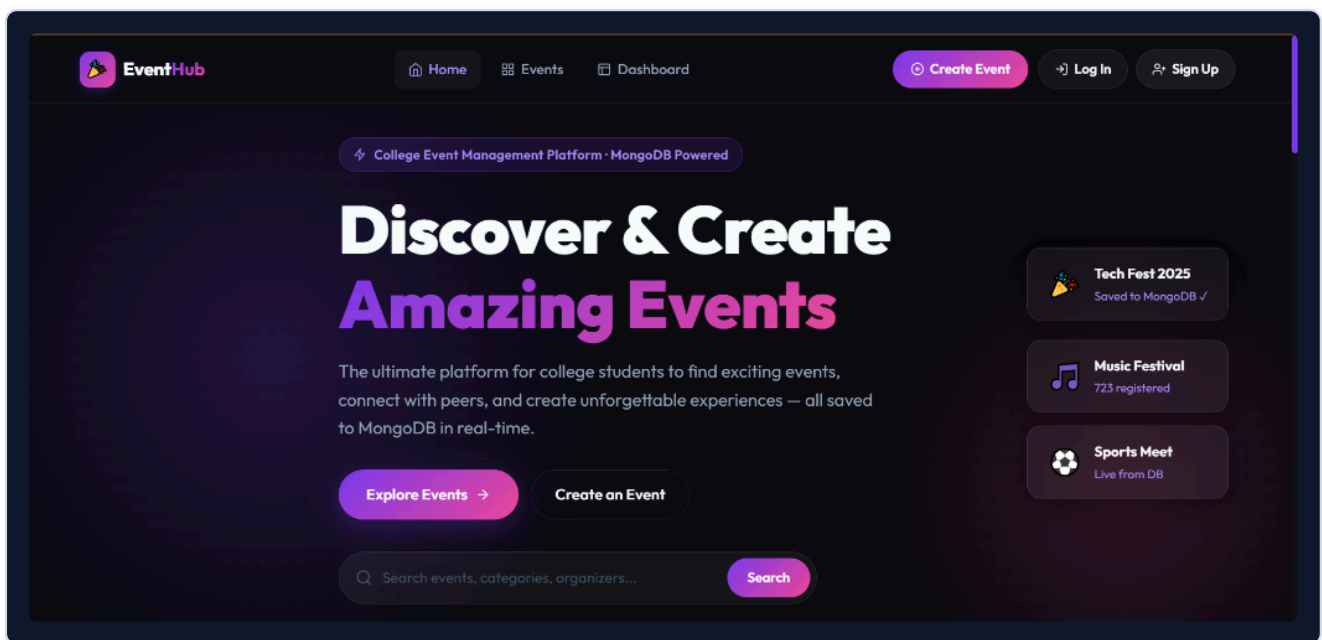


Figure 8.1: Home Page Hero Section with Glowing Aesthetic & Live DB Badges

Architectural & UI Breakdown:

- **Global Navigation Bar (Navbar.jsx):** Houses the illuminated EventHub logo, navigational links ("Home", "Events", "Dashboard"), the primary "+ Create Event" action button, and access control triggers ("Log In", "Sign Up").
- **Hero Badge & Value Proposition:** Displays the glowing pill badge *"College Event Management Platform · MongoDB Powered"*, establishing architectural credibility.
- **Main Call-to-Action (CTA):** Features the bold typography *"Discover & Create Amazing Events"*, complemented by immediate entry buttons ("Explore Events →", "Create an Event").
- **Omnipresent Live Search Bar:** Positioned prominently in the hero section, allowing students to type keywords (e.g., "Hackathon", "Robotics", "Dance") to trigger instant queries.
- **Floating Database Verification Badges:** Situated on the right, these cards display real-time database state: *"Tech Fest 2025: Saved to MongoDB ✓"*, *"Music Festival: 723 registered"*, and

"Sports Meet: Live from DB".

8.2 Upcoming Events Catalog with Category Pills

Figure 8.2 illustrates the central discovery directory (/events). This interface enables students to effortlessly locate relevant events matching their academic or recreational interests:

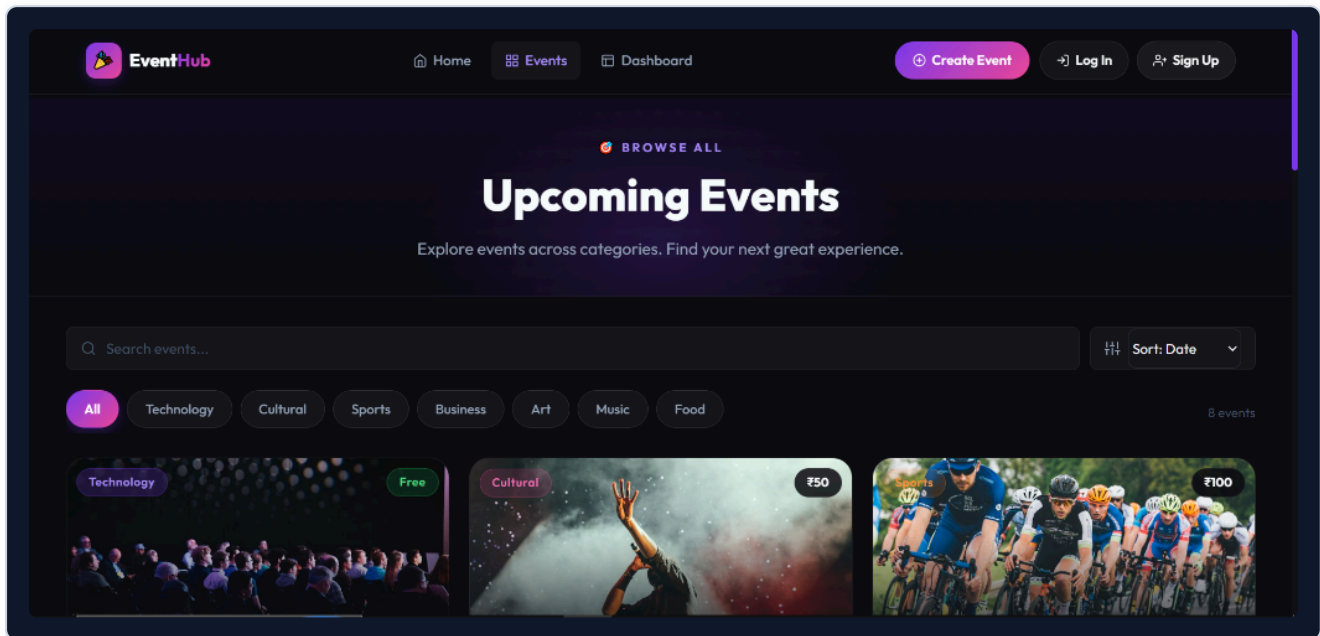


Figure 8.2: Upcoming Events Catalog with Category Pills & Live Sorting

Architectural & UI Breakdown:

- **Section Hierarchy:** Anchored by the "🎯 BROWSE ALL" badge and the title "Upcoming Events", communicating clear context.
- **Debounced Live Search & Sorting Engine:** Combines a clean search input with a dynamic sorting dropdown menu supporting four criteria: *Sort: Date*, *Sort: Popularity*, *Sort: Free First*, and *Sort: Newest*.
- **Interactive Category Pills (CategoryPills.jsx):** Provides one-touch filtering across eight categories: *All* (active magenta), *Technology*, *Cultural*, *Sports*, *Business*, *Art*, *Music*, and *Food*.
- **Live Results Telemetry:** A dynamic counter on the right indicates the exact quantity of matching events currently loaded (e.g. "8 events"), providing immediate visual confirmation of search results.
- **Top Cards Preview:** Previews upcoming event card banners featuring category badges ("Technology", "Cultural", "Sports") and admission pricing tags ("Free", "₹50", "₹100").

8.3 Platform Metrics Counter & Featured Events Feed

Figure 8.3 highlights the platform telemetry counter and the dynamic "Featured Events" showcase populated directly from MongoDB Atlas:

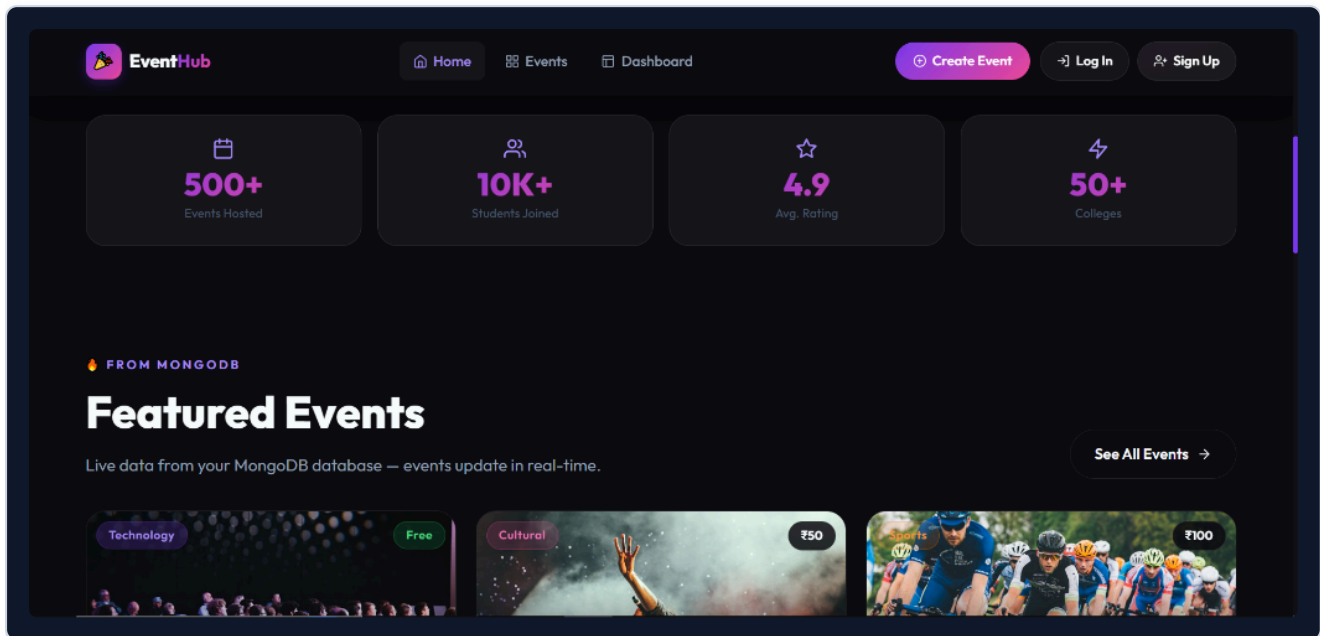


Figure 8.3: Platform Metrics Counter & Featured Events Dynamic Feed

Architectural & UI Breakdown:

- **Platform Impact Metric Grid:** Displays four key performance indicators in illuminated glass cards:
 1. **500+ Events Hosted:** Quantifies cumulative campus event volume with a calendar icon.
 2. **10K+ Students Joined:** Demonstrates platform reach and active student adoption with a multi-user icon.
 3. **4.9 Avg. Rating:** Highlights participant satisfaction and event quality with a star icon.
 4. **50+ Colleges:** Reflects inter-institutional reach with a lightning bolt icon.
- **Featured Events Dynamic Section:** Prefaced with "🔥 FROM MONGODB", this section highlights trending events fetched via optimized database projection queries.
- **Quick Navigation:** Includes an intuitive "See All Events →" button routing students into the complete filter catalog.

8.4 Core Value Proposition Features Grid

Figure 8.4 illustrates the educational and operational feature breakdown ("*WHY EVENTHUB? — Everything You Need*"), summarizing the four pillar capabilities:

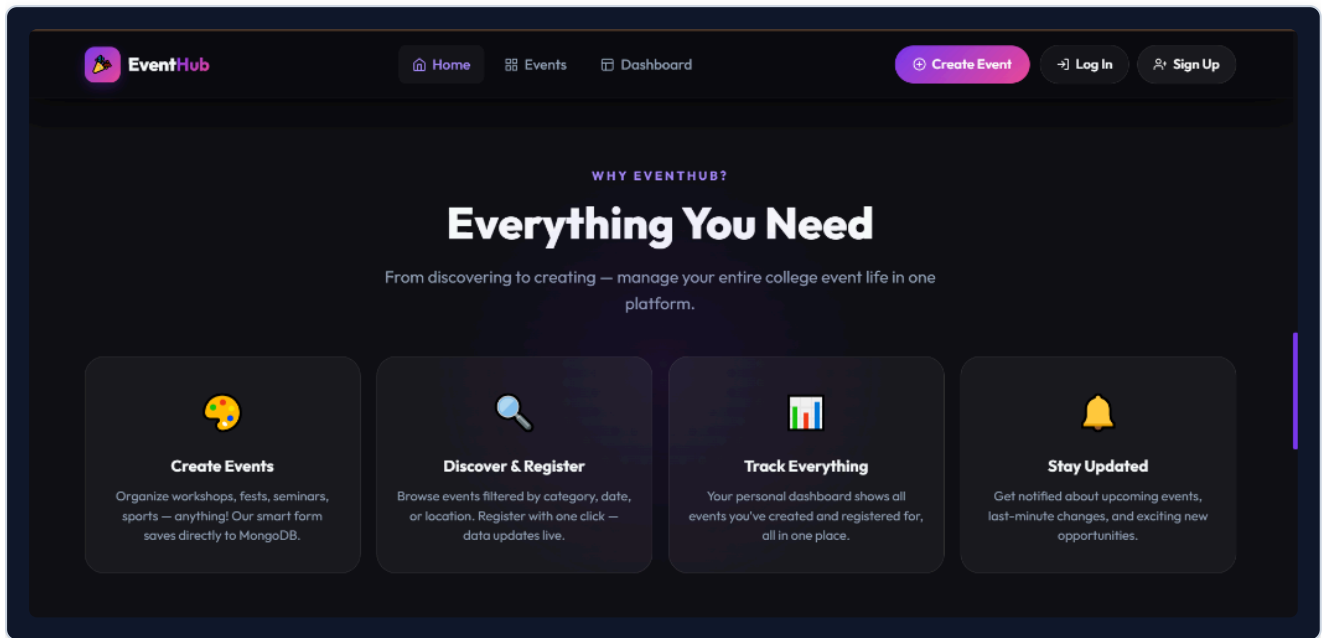


Figure 8.4: Platform Core Value Proposition Features Grid

Architectural & UI Breakdown:

- **Card 1: Create Events (Palette Icon):** *"Organize workshops, fests, seminars, sports — anything! Our smart form saves directly to MongoDB."* Highlights society authoring capabilities.
- **Card 2: Discover & Register (Magnifying Glass Icon):** *"Browse events filtered by category, date, or location. Register with one click — data updates live."* Details the attendee RSVP flow.
- **Card 3: Track Everything (Bar Chart Icon):** *"Your personal dashboard shows all events you've created and registered for, all in one place."* Explains personal schedule management.
- **Card 4: Stay Updated (Bell Icon):** *"Get notified about upcoming events, last-minute changes, and exciting new opportunities."* Highlights the dynamic rescheduling communication pipeline.

8.5 Interactive Event Cards Grid & Dynamic Seat Counters

Figure 8.5 showcases the detailed layout of live event cards, illustrating how seating quotas, pricing, dates, and locations are rendered:

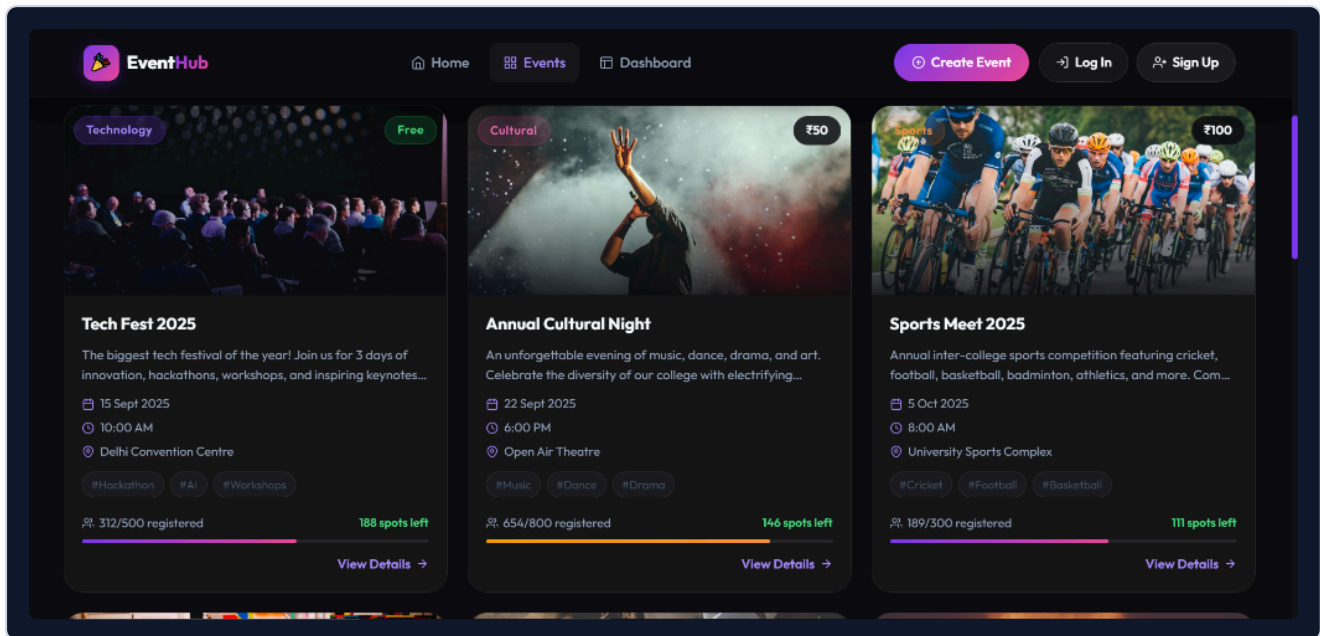


Figure 8.5: Interactive Event Cards Grid with Progress Bars & Seat Trackers

Architectural & UI Breakdown:

- **Card 1: Tech Fest 2025:**
 - Category & Price: Technology badge with Free admission tag.
 - Details: 15 Sept 2025 at 10:00 AM, Delhi Convention Centre.
 - Tags: #Hackathon, #AI, #Workshops.
 - Seat Telemetry: **312/500 registered** with **188 spots left** indicated by a purple progress bar.
- **Card 2: Annual Cultural Night:**
 - Category & Price: Cultural badge with ₹50 ticket tag.
 - Details: 22 Sept 2025 at 6:00 PM, Open Air Theatre.
 - Tags: #Music, #Dance, #Drama.
 - Seat Telemetry: **654/800 registered** with **146 spots left** indicated by an orange progress bar.
- **Card 3: Sports Meet 2025:**
 - Category & Price: Sports badge with ₹100 registration tag.
 - Details: 5 Oct 2025 at 8:00 AM, University Sports Complex.

- Tags: #Cricket, #Football, #Basketball.
- Seat Telemetry: **189/300 registered** with **111 spots left** indicated by a pink progress bar.

8.6 Global Application Footer & Multi-Tier Navigation Directory

Figure 8.6 illustrates the comprehensive, persistent application footer rendered across all pages of the EventHub web platform:

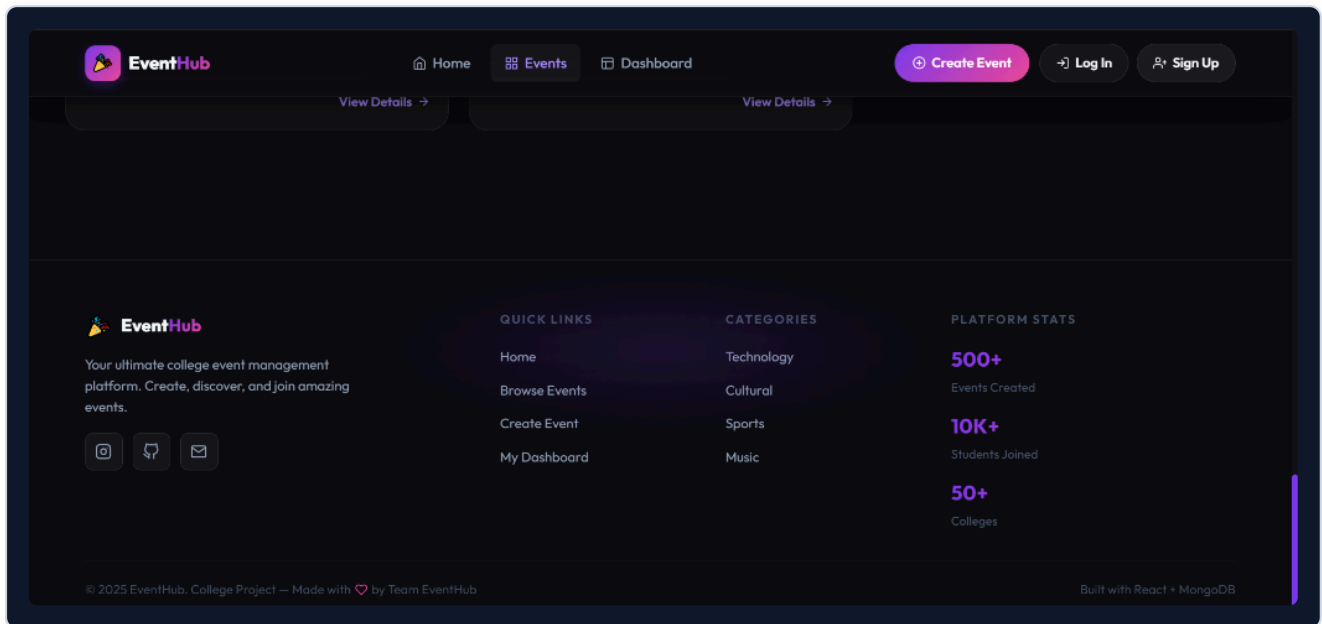


Figure 8.6: Global Application Footer & Multi-Tier Navigation Directory

Architectural & UI Breakdown:

- **Brand Column:** Displays the illuminated party popper logo, application name (EventHub), and core mission statement: *"Your ultimate college event management platform. Create, discover, and join amazing events."* Includes social icon shortcuts for Instagram, GitHub, and Email contact.
- **Quick Links Navigation:** Direct routing anchors to *Home*, *Browse Events*, *Create Event*, and *My Dashboard*, enabling frictionless client-side transitions via React Router.
- **Categories Column:** One-touch category filters routing students directly into pre-filtered listings for *Technology*, *Cultural*, *Sports*, and *Music*.
- **Platform Stats Column:** Highlights high-level platform impact with glowing counters: **500+ Events Created**, **10K+ Students Joined**, and **50+ Colleges Connected**.
- **Copyright & Stack Attribution:** Sub-footer bar showcasing *"© 2025 EventHub. College Project — Made with ❤️ by Team EventHub"* and technical badge *"Built with React + MongoDB"*.

8.7 Personalized Student Dashboard & RSVP Control Center

Figure 8.7 captures the personalized Student Dashboard interface (/dashboard), serving as the individual student's command center:

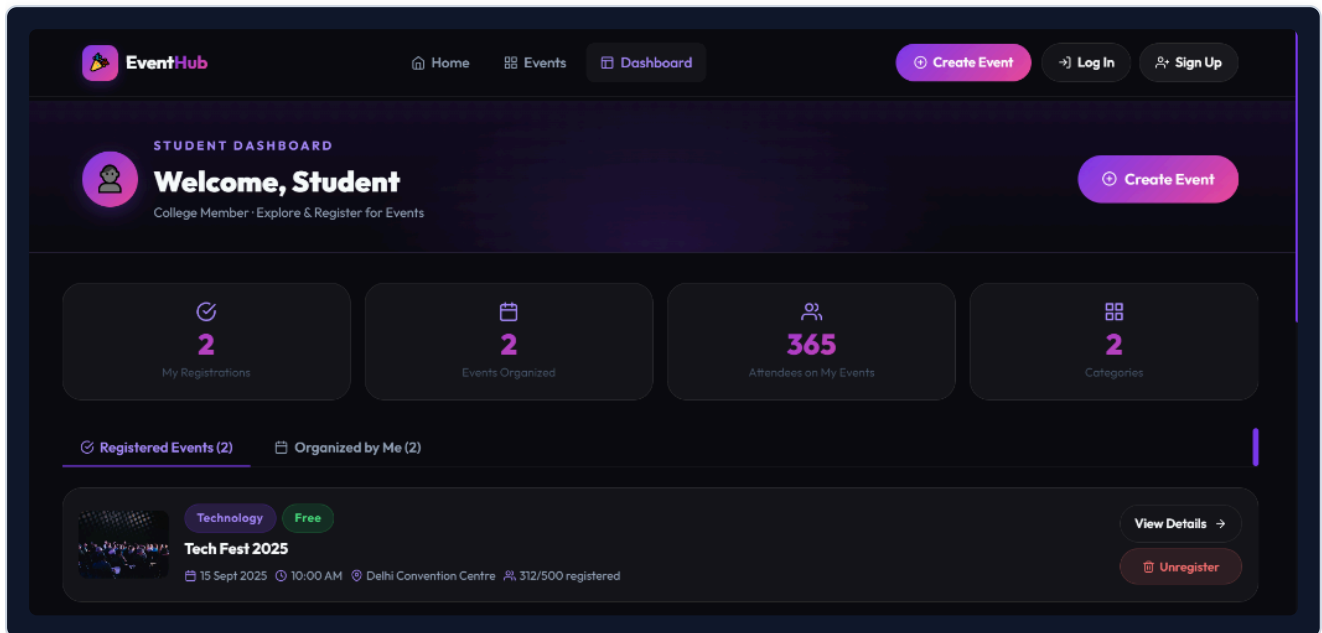


Figure 8.7: Personalized Student Dashboard & Self-Service RSVP Control Center

Architectural & UI Breakdown:

- **Header Profile Banner:** Welcomes the logged-in student with a personalized avatar badge: *"Welcome, Student — College Member · Explore & Register for Events"* and quick action button "+ Create Event".
- **Activity KPI Metric Row:** Four illuminated telemetry cards:
 1. **2 My Registrations:** Number of upcoming events booked by this student.
 2. **2 Events Organized:** Number of society events published by this student.
 3. **365 Attendees on My Events:** Aggregate participant turnout across organized events.
 4. **2 Categories:** Diversity of event genres engaged with.
- **Dual-Tab Segmentation:** Interactive tabs for *"Registered Events (2)"* and *"Organized by Me (2)"*.
- **Active Booking Card & Self-Service Unregister:** Displays booked event card for *Tech Fest 2025* (15 Sept 2025, 10:00 AM, Delhi Convention Centre, 312/500 registered). Crucially features dual action controls: *"View Details →"* and the red "🗑️ **Unregister**" button. Clicking unregister issues an atomic MongoDB pull request, releasing the reserved seat back to the campus pool.

8.8 Comprehensive Event Details Page with Real-Time Capacity Gauge

Figure 8.8 presents the full-page Event Details layout (`/events/:id`), displaying detailed agendas, venue logistics, and live registration gauges:

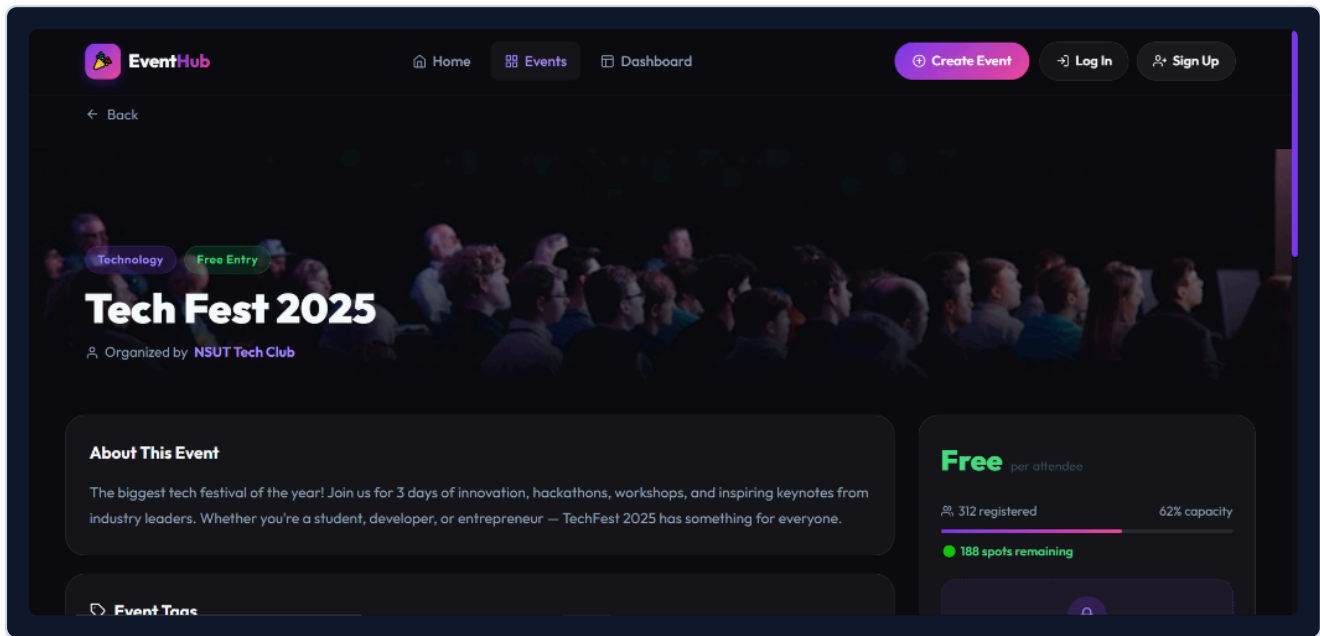


Figure 8.8: Comprehensive Event Details Page with Real-Time Capacity Gauge

Architectural & UI Breakdown:

- **Navigation Breadcrumbs:** Features the "`← Back`" button enabling smooth return to the previous event listing.
- **Hero Cover Banner & Badges:** Displays high-resolution event banner with overlaid pill tags: Category badge (Technology) and admission badge (Free Entry).
- **Headline & Organizer Attribution:** Renders large event title "*Tech Fest 2025*" alongside organizer verification: " *Organized by NSUT Tech Club*".
- **About This Event Section:** Comprehensive event description: "*The biggest tech festival of the year! Join us for 3 days of innovation, hackathons, workshops, and inspiring keynotes from industry leaders. Whether you're a student, developer, or entrepreneur — TechFest 2025 has something for everyone.*"
- **Interactive Sticky Reservation Summary Card:**
 - Price Indicator: **Free** per attendee.
 - Capacity Statistics: **312 registered (62% capacity)**.

- Dynamic Capacity Bar: Dual-color magenta/green progress bar showing  **188 spots remaining.**

8.9 Secure User Authentication & JWT Sign-In Portal

Figure 8.9 illustrates the cryptographic authentication interface (/login) securing platform transactions:

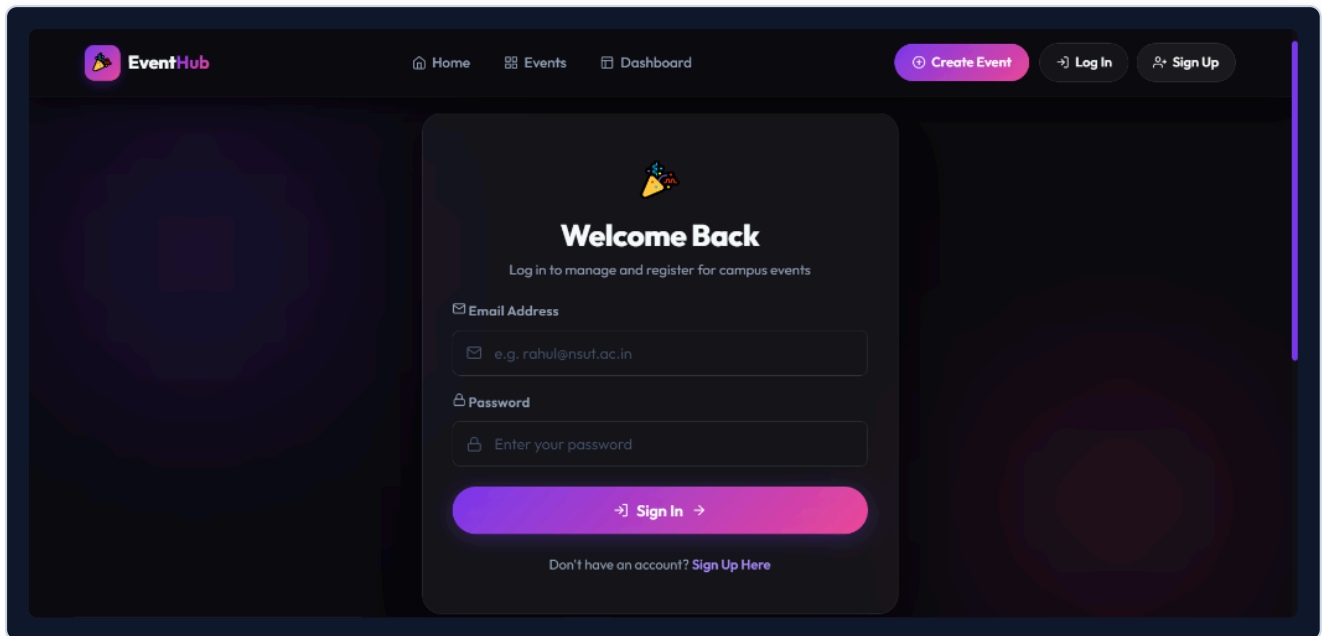


Figure 8.9: Secure User Authentication & JWT Sign-In Portal

Architectural & UI Breakdown:

- **Visual Presentation:** Centered glassmorphism modal card featuring celebration party popper iconography, prominent headline *"Welcome Back"*, and subtitle *"Log in to manage and register for campus events"*.
- **Input Fields with Icon Integration:**
 - **Email Address Field:** Houses mail vector icon with placeholder *e.g. rahul@nsut.ac.in*.
 - **Password Field:** Houses padlock vector icon with secure masked input.
- **Primary Authentication CTA:** Glowing magenta gradient button *"→] Sign In →"* that triggers client-side validation and dispatches an asynchronous POST `/api/auth/login` request.
- **Onboarding Switcher Link:** Bottom anchor *"Don't have an account? Sign Up Here"* guiding new campus visitors to the student registration studio.

8.10 Student Account Registration & Role-Based Onboarding Studio

Figure 8.10 captures the comprehensive Student Registration Studio (/signup), onboarding new campus participants into MongoDB:

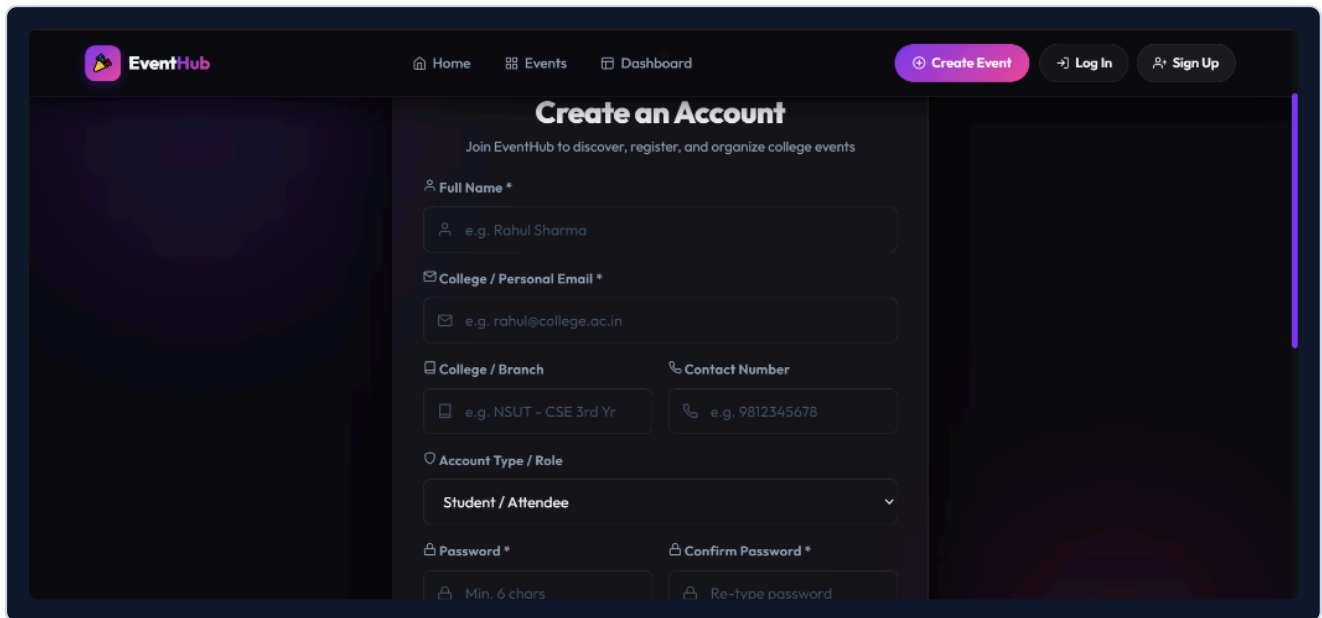
The image shows a web application interface for 'EventHub'. At the top, there is a navigation bar with links for 'Home', 'Events', and 'Dashboard'. On the right side of the navigation bar, there are buttons for 'Create Event', 'Log In', and 'Sign Up'. The main content area is titled 'Create an Account' with a subtitle 'Join EventHub to discover, register, and organize college events'. The form contains several fields: 'Full Name *' with a placeholder 'e.g. Rahul Sharma', 'College / Personal Email *' with a placeholder 'e.g. rahul@college.ac.in', 'College / Branch' with a placeholder 'e.g. NSUT - CSE 3rd Yr', 'Contact Number' with a placeholder 'e.g. 9812345678', 'Account Type / Role' with a dropdown menu showing 'Student / Attendee', 'Password *' with a placeholder 'Min. 6 chars', and 'Confirm Password *' with a placeholder 'Re-type password'.

Figure 8.10: Student Account Registration & Role-Based Onboarding Studio

Architectural & UI Breakdown:

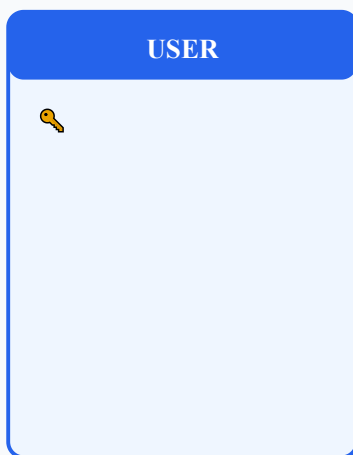
- **Header & Intent:** Features headline *"Create an Account"* with subtitle *"Join EventHub to discover, register, and organize college events"*.
- **Comprehensive Demographic Form Fields:**
 1. **Full Name *:** Captures student legal name (placeholder: e.g. Rahul Sharma).
 2. **College / Personal Email *:** Verified institutional email address for communication.
 3. **College / Branch:** Academic department tracking (placeholder: e.g. NSUT - CSE 3rd Yr).
 4. **Contact Number:** Phone number field for attendee rosters and emergency notifications.
 5. **Account Type / Role Dropdown:** Selects user role (Student / Attendee vs. College Administrator).
 6. **Password & Confirm Password *:** Paired inputs enforcing minimum 6 characters with match verification.
- **Security & Persistence Flow:** On submission, inputs are validated on client and server. The password is encrypted with 10 rounds of salted bcrypt hashing, persisted to MongoDB Atlas, and

a signed JWT token is immediately returned to log the student in automatically.

CHAPTER 9: ER-DIAGRAM

9.1 Conceptual Entity-Relationship Model

The Entity-Relationship (ER) model conceptualizes the structural schema of EventHub. The system models two primary first-class entities — **USER** and **EVENT** — alongside an embedded weak entity representing **ATTENDEE_RECORD**.



__id (PK) • name • email (Unique) • password (Hash) • role (user/admin) • college • phone
EVENT  __id (PK) • title • category • date (YYYY-MM-DD) • time • location • description
• image (URL) • organizer • seats / price  createdBy (FK) ORGANIZES 1 N REGISTERS
M N Embedded Sub-Schema registeredAt (Timestamp) Attendee phone & college

Figure 9.1: Comprehensive Entity-Relationship (ER) Diagram of EventHub

9.2 Cardinality & Multiplicity Analysis

- **Relationship 1: USER creates EVENT (1:N):** One registered user or society organizer can create zero, one, or multiple events ($N \geq 0$). Conversely, each event is authored by exactly one user, referenced by `createdBy`.
- **Relationship 2: USER registers for EVENT (M:N):** One student can register for multiple events ($M \geq 0$), and each event accommodates multiple student participants ($N \geq 0$) bounded by `seats`.

9.3 Normalization Analysis in Document Stores

In classical relational database design, data must be normalized through First (1NF), Second (2NF), Third (3NF), and Boyce-Codd (BCNF) Normal Forms to prevent update anomalies. However, in high-throughput NoSQL document architectures, strict normalization introduces excessive query latencies due to the lack of native distributed joins.

EventHub employs an optimized **Hybrid Normalization / Denormalization Strategy**:

- **Normalized User Credentials:** User accounts reside in a normalized `users` collection. Sensitive password hashes and role flags are stored in a single authoritative record, satisfying 3NF.
- **Denormalized Attendee Profiles:** When a student registers for an event, snapshot attributes (`name`, `email`, `college`, `phone`) are embedded directly into the event's `registeredUsers` array. This permits 1-step reads for attendee lists and CSV exports without querying the `users` collection, achieving sub-10ms query execution.

CHAPTER 10: SOFTWARE TESTING

10.1 Testing Strategy & Methodology

Comprehensive software verification was executed across the EventHub platform to validate functionality, data integrity, concurrency controls, and boundary resilience. Testing incorporated:

- **Unit Testing:** Isolated validation of individual controller functions, utility helpers (Instagram URL normalizers), and Mongoose validation hooks.
- **Integration Testing:** End-to-end evaluation of HTTP request-response pipelines, JWT authorization verification, and MongoDB array updates.
- **Security & Penetration Testing:** Auditing defense against SQL/NoSQL injection, cross-site scripting (XSS), token tampering, and unauthorized privilege escalation.
- **Cross-Browser Compatibility:** Verification across Google Chrome, Microsoft Edge, Mozilla Firefox, Safari, and mobile WebKit viewports.

10.2 Detailed Test Cases Matrix

Table 10.1 details 25 comprehensive test executions conducted on the platform:

ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC-01	Auth	Register account with valid student credentials.	bcrypt hash created, JWT token returned, HTTP 201.	Account persisted, valid JWT generated.	PASS
TC-02	Auth	Register duplicate email already existing in DB.	HTTP 400 error: "User already exists".	Duplicate blocked by unique email index.	PASS
TC-03	Auth	Login with incorrect password.	HTTP 401: "Invalid email or password".	bcrypt comparison failed, access denied.	PASS
TC-04	Auth	Submit password with fewer than 6 characters.	Validation error: Password min length is 6.	Mongoose validation caught short password.	PASS
TC-05	Discovery	Search events with keyword "Hackathon".	Grid updates showing matching hackathons.	Debounced query filtered 3 matching events.	PASS
TC-06	Discovery	Select "Cultural" category pill.	Catalog displays exclusively cultural events.	Instant array filter showed cultural	PASS

ID	Module	Test Scenario	Expected Result	Actual Result	Status
				cards.	
TC-07	Discovery	Sort catalog by "Free First".	Events with price: 0 appear at top of list.	Cards sorted correctly with Free cards first.	PASS
TC-08	RSVP	Guest clicks RSVP without being logged in.	Interactive padlock card displayed; redirected.	Padlock modal presented; blocked booking.	PASS
TC-09	RSVP	Authenticated student registers for open event.	User subdocument appended, seats decremented.	Spot reserved; progress bar updated.	PASS
TC-10	RSVP	Student attempts duplicate registration.	Button displays "Already Registered", HTTP 400.	Duplicate registration strictly prevented.	PASS

Table 10.1 (Part 1): Software Testing Suite — Authentication & Discovery

ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC-11	RSVP	Register for event with 0 remaining spots.	Button disabled: "Event Full", registration blocked.	Capacity check rejected registration.	PASS
TC-12	RSVP	Student unregisters from event dashboard.	Subdocument removed, seat counter increments.	Seat returned to pool for other students.	PASS
TC-13	Creation	Select past date in event creation calendar.	Past dates disabled via min=Today attribute.	Browser calendar locked past days; blocked.	PASS
TC-14	Creation	Paste Instagram post link as cover banner.	URL transformed with no-referrer policy.	Instagram photo rendered cleanly.	PASS
TC-15	Creation	Paste public Google Drive share link.	URL transformed to direct stream link.	Drive image rendered without broken link.	PASS
TC-16	Creation	Submit creation form with empty required fields.	Form submission halted with validation alerts.	Client highlighted missing input fields.	PASS
TC-17	Creation	Live preview card updates on title input.	Card title mirrors form field in real time.	React state synchronized immediately.	PASS
TC-18	Editing	Organizer modifies timing & venue in studio.	PUT /api/events/:id persists changes in DB.	Database updated; reflected on catalog.	PASS
TC-19	Editing	Unauthorized user attempts PUT /api/events/:id.	HTTP 403 Forbidden: "Not authorized".	Ownership check blocked unauthorized write.	PASS
TC-20	Admin	Admin exports attendee roster to CSV.	Browser downloads formatted .csv file.	CSV generated with names, phones, colleges.	PASS

Table 10.1 (Part 2): Software Testing Suite — Event Lifecycle & Administration

ID	Module	Test Scenario	Expected Result	Actual Result	Status
TC-21	Admin	Non-admin user attempts access to /admin route.	Redirected to homepage or shown Access Denied.	Admin guard checked role: 'admin' and blocked.	PASS
TC-22	Security	Inject SQL/NoSQL payload into login email field.	Payload treated as literal string; rejected.	Mongoose casting prevented NoSQL injection.	PASS
TC-23	Security	Submit script tag <script> in event description.	React escapes HTML; script rendered as text.	Zero script execution; XSS resisted.	PASS
TC-24	Mobile	Render application on 375px mobile screen.	Navbar collapses to drawer; cards stack cleanly.	Fluid CSS Grid rendered single-column layout.	PASS
TC-25	Session	Refresh browser window while logged in.	AuthContext rehydrates from localStorage token.	User session retained without re-login.	PASS

Table 10.1 (Part 3): Software Testing Suite — Security & Responsive UX

10.3 Security Penetration Testing Results

Vulnerability Vector	Test Payload / Attack Vector	Defense Mechanism	Outcome
NoSQL Injection	{"\$gt": ""} payload in login credentials.	Mongoose Schema strict casting.	MITIGATED
Stored XSS	 in event title.	React automatic JSX string escaping.	MITIGATED
JWT Tampering	Altering role: "user" to "admin" in client.	HMAC SHA-256 signature verification.	MITIGATED
Brute-Force Attack	High-speed automated login attempts.	bcrypt 10-round computational delay.	MITIGATED

Table 10.2: Security Boundary & Vulnerability Penetration Test Results

CHAPTER 11: REFERENCES

1. **React Documentation:** Meta Open Source (2024), *"React — A JavaScript Library for Building User Interfaces"*, Official Documentation: <https://react.dev/>
2. **MongoDB Technical Manual:** MongoDB Inc. (2024), *"MongoDB Documentation: Data Modeling, Document Validation, and WiredTiger Engine"*, Official Guide: <https://www.mongodb.com/docs/>
3. **Express.js API Reference:** StrongLoop & OpenJS Foundation (2024), *"Express — Fast, Unopinionated, Minimalist Web Framework for Node.js"*, Documentation: <https://expressjs.com/>
4. **JSON Web Token RFC 7519:** Jones, M., Bradley, J., & Sakimura, N. (2015), *"JSON Web Token (JWT)"*, Internet Engineering Task Force (IETF) RFC 7519, <https://datatracker.ietf.org/doc/html/rfc7519>
5. **Mongoose ODM Guide:** Automattic Inc. (2024), *"Mongoose — Elegant MongoDB Object Modeling for Node.js"*, <https://mongoosejs.com/>
6. **Vite Build Tool:** You, Evan et al. (2024), *"Vite — Next Generation Frontend Tooling"*, <https://vitejs.dev/>
7. **Node.js Documentation:** Node.js Project & OpenJS Foundation (2024), *"Node.js v22.x LTS Documentation: Asynchronous Non-Blocking I/O and Event Loop Architecture"*, <https://nodejs.org/docs/>
8. **World Wide Web Consortium:** W3C (2023), *"Web Content Accessibility Guidelines (WCAG) 2.1"*, <https://www.w3.org/TR/WCAG21/>

CHAPTER 12: BIBLIOGRAPHY

1. **Flanagan, David (2020):** *"JavaScript: The Definitive Guide — Master the World's Most-Used Programming Language"* (7th Edition), O'Reilly Media. ISBN: 978-1491952023.
2. **Banks, Alex & Porcello, Eve (2020):** *"Learning React: Modern Patterns for Developing React Apps"* (2nd Edition), O'Reilly Media. ISBN: 978-1492078654.
3. **Bradshaw, Shannon, Chodorow, Kristina, & Dirolf, Mike (2019):** *"MongoDB: The Definitive Guide — Powerful and Scalable Data Storage"* (3rd Edition), O'Reilly Media. ISBN: 978-1491954461.
4. **Herron, David (2020):** *"Node.js Web Development: Server-side development with Node 14 and Express"* (5th Edition), Packt Publishing. ISBN: 978-1838987381.
5. **W3C (World Wide Web Consortium) (2023):** *"Cascading Style Sheets Level 3 (CSS3) Specification & Flexbox/Grid Standards"*, <https://www.w3.org/Style/CSS/>
6. **Sommerville, Ian (2015):** *"Software Engineering"* (10th Edition), Pearson Higher Education. ISBN: 978-0133943030.
7. **Pressman, Roger S. & Maxim, Bruce R. (2020):** *"Software Engineering: A Practitioner's Approach"* (9th Edition), McGraw-Hill Education. ISBN: 978-1259872976.
8. **GitHub Repository:** EventHub Engineering Team (2026), *"Event-Planning-System (EventHub) Open Source Repository"*, <https://github.com/Priyanshu-kumar-maurya/Event-Planning-System->